



RV College of
Engineering®

Artificial Intelligence and Machine learning

IS353IA

UNIT 4

Saraswathi G Datar

Assistant Professor

Department of Computer Science and Engineering

RVCE

All material presented in this PowerPoint is credited to the respective authors of standard textbooks, reference presentations shared by senior faculty at **RVCE**, and authentic internet sources.

Note to Students:

This presentation is meant **only for classroom reference and conceptual understanding**.

For exam and detailed preparation, please refer to the **textbook extracts and materials shared in Google Classroom**, as the content here is a **summary version** and not exhaustive.

Go, change the world®



RV College of
Engineering®

- **Nearest Neighbor Classifiers:** Characteristics of Nearest Neighbour Classifiers.
- **Naive Bayes Classifier:** Basics of Probability Theory, Naive Bayes assumption.
- **Logistic Regression:** Logistic Regression as a Generalized Linear Model, Learning Model Parameters, Characteristics of Logistic Regression.
- **Ensemble Methods:** Methods for constructing Ensemble classifiers, Bagging, Boosting, Random Forests.

Go, change the world®



RV College of
Engineering®

Nearest Neighbor Classifiers: Characteristics of Nearest Neighbour Classifiers.

Go, change the world®



Model-Based Learning:

- Builds a **generalized model** from training data (rules, equations).
- Learns an **approximation of target function**.
- Once trained, predicts **without referring to training data**.
- **Example:** Spam filter learns rules to detect spam (e.g., suspicious words, unknown links).

Instance-Based Learning:

- **Does not build a model.** Retains all training data in memory.
- **Learning happens at prediction time.**
- Compares new input with stored instances using a **distance/similarity measure**.
- **Flexible but computationally expensive.**



Key Features of Instance-Based Learning

- Memorizes all training examples → hence **memory-based**.
- Uses **similarity measures** (e.g., Euclidean distance) to find closest examples.
- Makes decisions based on **nearest or most similar instances**.
- Lazy learner** → delays learning until a new query arrives.
- No generalization rule** is created.

Example

- For a new email, the system checks similarity with stored spam/not-spam examples before deciding.



Classification Framework

• Involves two steps:

- **Inductive step:** Build a classification model from data.
- **Deductive step:** Apply model to classify test examples.

• **Eager learners (e.g., Decision Trees, Rule-based):**

- Build model immediately after training data is available.
- Note: Rule-Based Learning: A type of model-based learning where the model is represented as a set of IF–THEN rules.

• **Lazy learners (e.g., k-NN):**

- Delay model building until a test example needs classification.

How it Works

Each instance = a point in **d-dimensional feature space** (d = number of attributes).

For a test instance:

- Compute **distance/proximity** (e.g., Euclidean distance) to all training points.
- Identify **k-nearest neighbors** (1-NN, 2-NN, 3-NN, etc.).
- **Majority class** among neighbors → predicted label.

Example

- **1-NN:** Closest single point decides class.
- **3-NN:** Majority among 3 closest points decides class.



A shared point of curiosity when understanding this algorithm is “Is it better to approximate a generic target function rather than estimating it separately for each instance?” Yes, using generalized prediction models as in model based learning is often more practical and effective than estimating the target function for each new instance individually. However, the choice depends on the problem domain and data characteristics. In instance-based learning methods, predictions are made by considering a small neighborhood of similar instances from the training data. Here, these methods don't learn one rule for the entire dataset, but build new rules everytime based on local data around. This approach captures the local structure and variations in the data, which may be missed by generalized estimations.

In which situations would a lazy learner outperform a model-based learner? Can you think of a domain where memorizing examples is more effective than building a model?

Think of a food delivery app like zomato or swiggy recommending dishes near you on your tour to Mumbai.

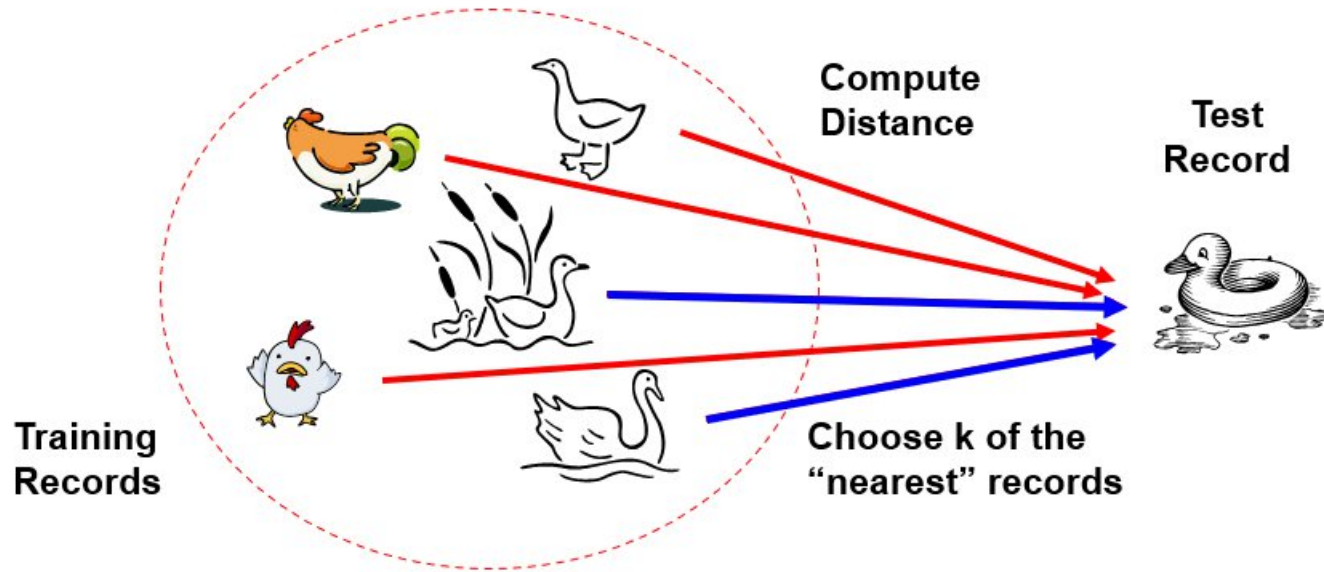


Nearest Neighbour Classifiers

Go, change the world®

Basic idea:

If it walks like a duck, quacks like a duck, then it's probably a duck

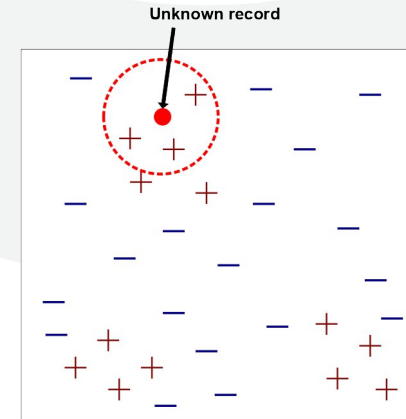


Nearest Neighbour Classifiers

Go, change the world®

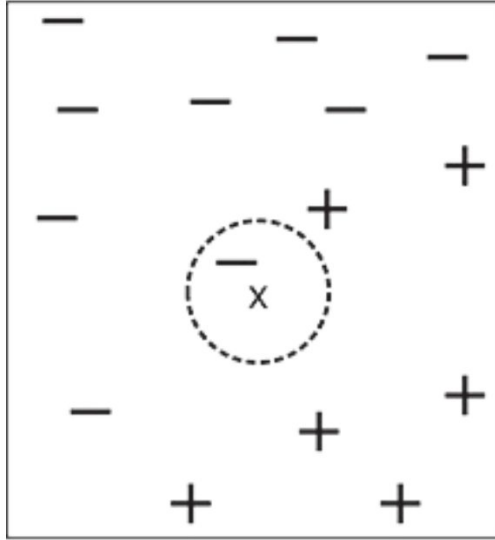
Requires the following:

- A set of labeled records
- Proximity metric to compute distance/similarity between a pair of records
e.g., Euclidean distance
- The value of k , the number of nearest neighbors to retrieve
- A method for using class labels of K nearest neighbors to determine the class label of unknown record (e.g., by taking majority vote)

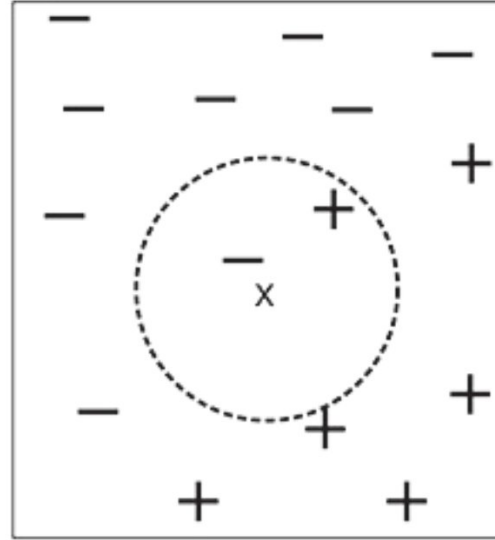




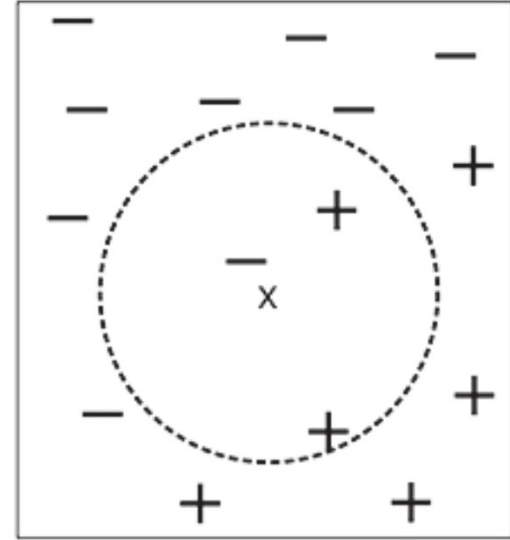
Nearest Neighbour Classifiers



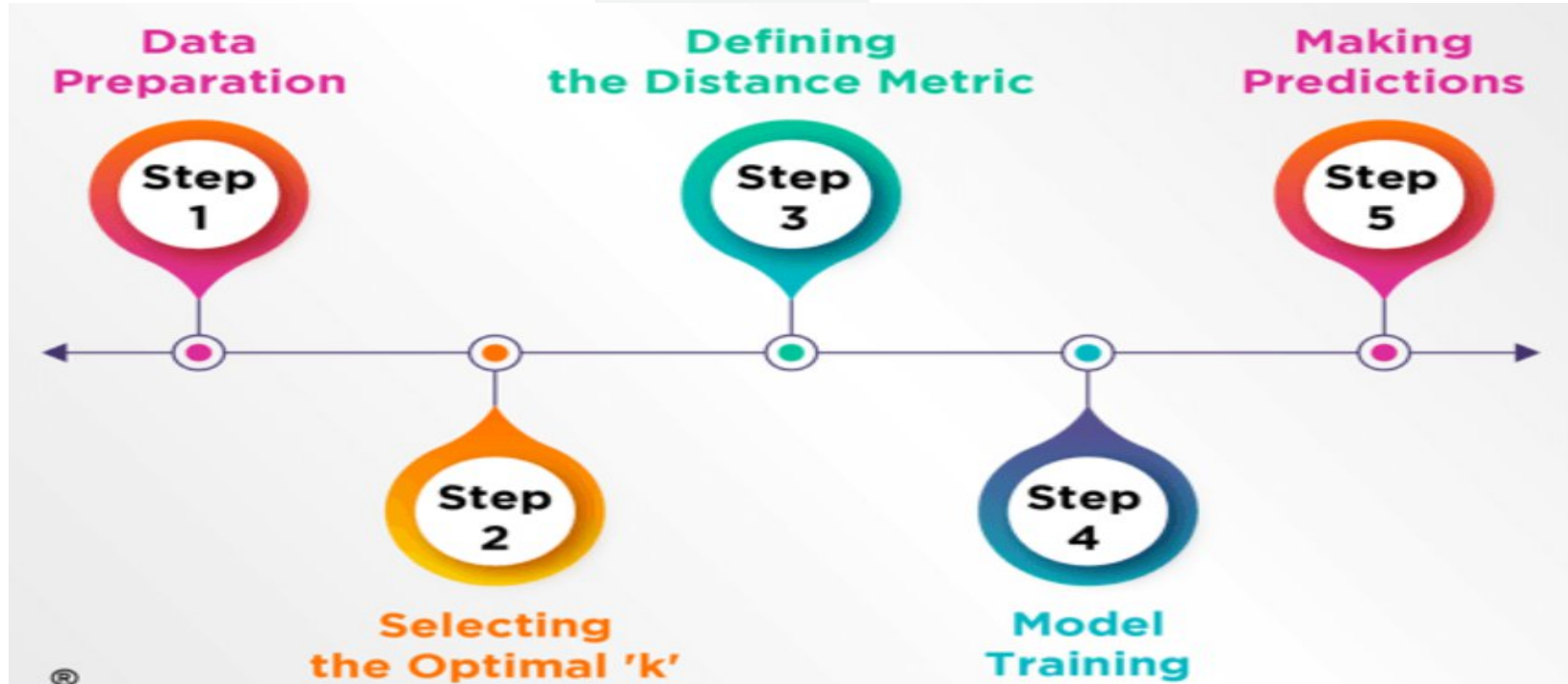
(a) 1-nearest neighbor



(b) 2-nearest neighbor



(c) 3-nearest neighbor





Characteristics of Nearest-Neighbor Classifiers (k-NN)

1. Instance-Based Learning

- Belongs to **instance-based learning** (no model abstraction).
- Uses **training instances directly** for prediction.
- Requires:
 - A **proximity/similarity measure** (e.g., Euclidean distance).
 - A **classification rule** (e.g., majority voting among nearest neighbors).

2. Lazy Learning Nature

- **No model-building phase** → minimal training time.
- **High prediction cost** → proximity must be computed for every test instance.
- Opposite of **eager learners** (e.g., decision trees), which train first and classify fast.

3. Local vs Global Decision

- Makes predictions using **local information** (neighbors of the test point).
- Decision trees/rule-based methods find **global patterns**.
- **Sensitive to noise**, especially for **small k** values.



Characteristics of Nearest-Neighbor Classifiers (k-NN)

4. Decision Boundaries

- Can form **complex, flexible, non-linear boundaries**.
- More adaptable than decision trees' rectilinear boundaries.
- **High variability** — depends on training data composition.
- Increasing **k** smooths the boundary (reduces variability).

5. Importance of Proper Distance Measure

- Accuracy depends on **choice of proximity metric** and **data scaling**.
- Attributes with different scales can distort distance (e.g., height in meters vs. weight in pounds).
- **Normalization/standardization** is essential.

6. Nearest neighbor classifiers have difficulty handling missing values in both the training and test sets since proximity computations normally require the presence of all attributes.



K-Nearest-Neighbor Classifiers (k- NN)

The simplest instance based learning method is K Nearest Neighbor algorithm . KNN is a non parametric algorithm meaning that it does not make assumptions about shape of data and learns from data directly.

Example: Predicting salary of a person based on experience

Let's consider the salary of a person who has 5 years of experience. In this case, KNN looks at all people with 4-6 years of experience and calculates the average of their salaries to make a prediction.

The KNN classifier works by using a labelled training dataset to identify the K nearest neighbours to a new data point and then votes on the most common class among these neighbours. Here's how it works:

1. Compute the distance from the new data point to each point in the training set.
2. Choose the K closest points based on these distances.
3. Identify the most common class among these K neighbours.
4. Assign this majority class to the new data point.



Ex: If we have a dataset of students with hours studied and marks scored in %, we want to classify a new student as Pass/Fail using KNN algorithm.

Study Hours	Marks Scored(%)	Class Labels
2	50	Fail
4	55	Fail
6	65	Pass
8	70	Pass
10	85	Pass



K-Nearest-Neighbor Classifiers (k- NN)

Go, change the world®

If the new data point is given as a student studying for 5 hours with marks scored 60%. What will be the predicted class for this student if K=3.

- 1) Using Euclidean distance to calculate distance between the new data point |5,60| to each point in the training set.

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Distance to |2,50|:

$$d = \sqrt{(5-2)^2 + (60-50)^2} = \sqrt{109} = 10.44$$

Distance to |4,55|:

$$d = \sqrt{(5-4)^2 + (60-55)^2} = \sqrt{26} = 5.10$$

Distance to |6,65|:

$$d = \sqrt{(5-6)^2 + (60-65)^2} = \sqrt{26} = 5.10$$

Distance to |8,70|:

$$d = \sqrt{(5-8)^2 + (60-70)^2} = \sqrt{109} = 10.44$$

Distance to |10,85|:

$$d = \sqrt{(5-10)^2 + (60-85)^2} = \sqrt{650} = 25.49$$



K-Nearest-Neighbor Classifiers (k- NN)

2) Sort the distances in ascending order

|4,55| : 5.10 - Fail

|6,65| : 5.10 - Pass

|2,50| : 10.44 - Fail

|8,70| : 10.44 - Pass

|10,85| : 25.49 - Pass

3) Choose K=3 closest neighbors

|4,55| : 5.10 - Fail

|6,65| : 5.10 - Pass

|2,50| : 10.44 - Fail

4) Find the majority class among these K neighbors

Fail - 2

Pass - 1

Majority class is Fail

k-NN is sensitive to tie-breaking when distances are equal.

The new student who studied for 5 hours and scored marks 60% is classified as Fail.



K-Nearest-Neighbor Classifiers (k- NN) Algorithm

Algorithm 5.2 The k -nearest neighbor classification algorithm.

- 1: Let k be the number of nearest neighbors and D be the set of training examples.
 - 2: **for** each test example $z = (\mathbf{x}', y')$ **do**
 - 3: Compute $d(\mathbf{x}', \mathbf{x})$, the distance between z and every example, $(\mathbf{x}, y) \in D$.
 - 4: Select $D_z \subseteq D$, the set of k closest training examples to z .
 - 5: $y' = \underset{v}{\operatorname{argmax}} \sum_{(\mathbf{x}_i, y_i) \in D_z} I(v = y_i)$
 - 6: **end for**
-



Distance Metrics

The idea behind KNN algorithm is based on the concept of “Nearness” or “Similarity”. The closer the new data point to the existing data, the more likely it is to belong to the same class as those nearby points.

To measure this nearness, we require some quantitative method, typically a distance metric. While there are many options to consider when we talk about distance calculation methods, **Euclidean Distance and Manhattan Distance** are widely used for KNN because of their simplicity, easy interpretation and its suitability to deal with numerical data.

The other distance metrics can be used in different context like **Cosine similarity** which is a better measurement metrics for problems that include higher dimensional sparse data or text data, **Hamming distance** which only works with binary or categorical data, **Chebyshev distance** which aids in board games or chess like grid problems, **Wasserstein (Earth Mover’s)** distance metrics which is used in Deep Learning concepts for comparing distributions(e.g, GANs), **Correlation distance** which can be used in time series or gene expression analysis to measure the dissimilarity, **Jaccard distance** that is used to measure the irregularities or dissimilarity in the sets most frequently used in the apps for recommendation system or clustering text data. The choice of right distance metrics impact performance of the algorithm.



(a) Euclidean Distance

Go, change the world®

The Euclidean distance is a calculation of the shortest possible length between two points which can be represented by a straight line if we connect them. The distance is always a non-negative number. The metric uses pythagorean theorem as its baseline to quantify the value.

The formula here depends on the number of dimensions. The features(columns) in the dataset are referred to as dimension here.

Each column defines one feature/dimension in the dataset. So if you have [“Age”, “Date-of-birth”], it is a 2D space. If it has three attributes, say [“Age”, “Height”, “Birthmark”], it becomes a 3D space. Depending on these numbers, we use different versions of Euclidean algorithm as below:

Two Dimensions (2D)

Given two points, (x_1, y_1) and (x_2, y_2) , the Euclidean distance d is given by:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$



(a) Euclidean Distance

Go, change the world®

Two Dimensions (2D)

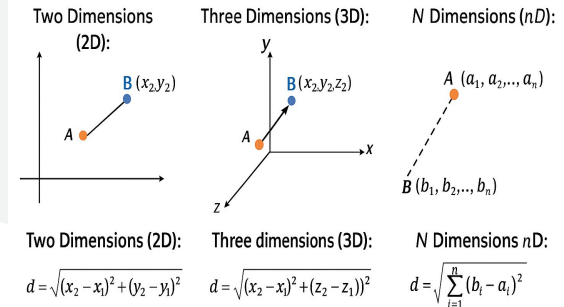
Given two points, (x_1, y_1) and (x_2, y_2) , the Euclidean distance d is given by:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Three Dimensions (3D)

Given two points, (x_1, y_1, z_1) and (x_2, y_2, z_2) , the Euclidean distance d is given by:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$$



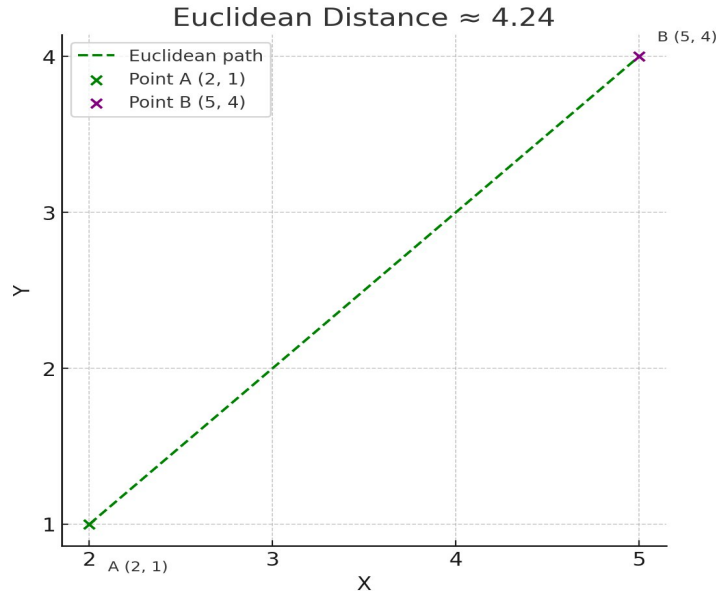


(a) Euclidean Distance

Go, change the world®

Here's the 2D graph showing Euclidean Distance between Point A (2, 1) and Point B (5, 4):

$$\text{Euclidean Distance} = \sqrt{(5 - 2)^2 + (4 - 1)^2} = \sqrt{18} \approx 4.24$$





(b) Manhattan Distance

Go, change the world®

(b) Manhattan Distance The Manhattan distance takes the coordinates of points and calculates the sum of the absolute difference between the corresponding coordinates. It analyses and measures the distance for grid-like paths.

The formula for different dimensions are listed below:

Two Dimensions (2D)

Given two points: $A = (x_1, y_1)$ and $B = (x_2, y_2)$

Manhattan Distance (D):

$$D = |x_2 - x_1| + |y_2 - y_1|$$



Three Dimensions (3D)

Given two points: $A = (x_1, y_1, z_1)$ and $B = (x_2, y_2, z_2)$

Manhattan Distance (D): $D = |x_2 - x_1| + |y_2 - y_1| + |z_2 - z_1|$

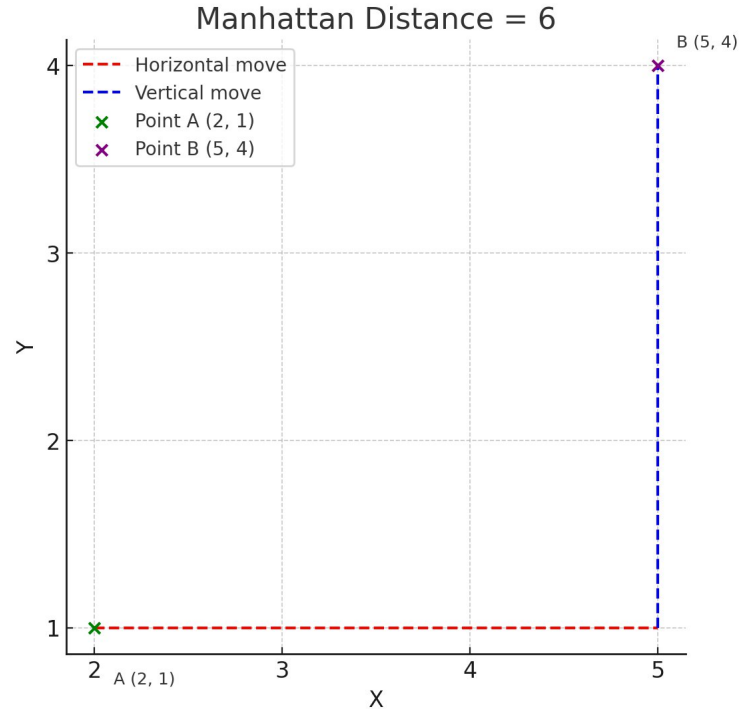
Imagine you're moving on a grid (like city streets) and you can only move **horizontally or vertically** — not diagonally. Let's visualize two points:

- Point A: (2, 1)
- Point B: (5, 4)
- Manhattan Distance = $|5 - 2| + |4 - 1| = 3 + 3 = 6$



(b) Manhattan Distance

Go, change the world®





How to Determine the Class Label of a Test Sample

When a new data point arrives:

1. Look through the **stored data in the memory unit** (called "instances") and check how similar each one is.
2. **Using this similarity, compare** new data to the closest matches and **predict** the output accordingly.

- **Find K nearest neighbors** – based on distance (like Euclidean, Manhattan, etc.).

- **Majority voting:**

- If most neighbors belong to “Class A”, the new sample is labeled as “Class A.”

- **Weighted voting (optional):**

- Closer points are more influential.

- Example:

means — if a neighbor is twice as far, it has only one-fourth the weight.

$$w = \frac{1}{d^2}$$



Why it matters:

The **distance metric** determines what “closeness” means.

Choosing the wrong metric can completely change classification outcomes.

Common examples:

- **Euclidean Distance:** Straight-line distance; best for continuous numeric data.
- **Manhattan Distance:** Grid-like distance (sum of absolute differences).
- **Cosine Similarity:** Measures **angle** between vectors, not magnitude — perfect for text or high-dimensional sparse data.
 - Used in **document classification**, **NLP**, and **recommendation systems**.

Example to explain:

Two documents might have similar *words* (content direction) but different *lengths*.

Cosine similarity sees them as similar (since the angle between their word vectors is small), while Euclidean might not.



Why preprocessing is critical:

KNN is **distance-based**, so features with large ranges dominate the calculation.

We need to make all features **comparable**.

Example:

- Height: 1.5 – 1.8
- Weight: 90 – 300
- Income: 10,000 – 1,000,000

If you use Euclidean distance, the difference in income overwhelms the height/weight — the algorithm becomes biased.

Solution:

- Normalization:** Rescale features to a common range (e.g., 0–1).
- Standardization (Z-score):** Transform data to have mean = 0 and SD = 1.
- For time-series:** Always standardize to prevent drift due to magnitude differences.

Analogy:

“It’s like comparing distances in meters and kilometers — unless you convert them to the same unit, your comparison is meaningless.”



Problem:

KNN needs complete data for distance computation.

If even one attribute is missing, the distance cannot be calculated correctly.

Possible approaches:

1.Imputation: Fill missing values using:

1. Mean / Median (for numeric data)
2. Mode (for categorical data)

2.Use partial attributes:

1. Compute distance only on attributes that exist in both instances.
2. But this causes inconsistency because each pair might use a different subset → distances aren't comparable.



Concept:

KNN treats **all features equally** in distance calculation.

So irrelevant or duplicated features can mislead the algorithm.

Problems:

- **Irrelevant features:** Add random noise → misclassification.
- **Redundant features:** Overemphasize certain patterns → bias in decision.

Example:

If you're classifying animals by size, color, and number of legs adding “serial number” as a feature adds *irrelevant noise*.

If you include both “weight” and “mass” — you've added *redundant information*.

Solution:

- **Feature Selection:** Use only discriminative features (e.g., mutual information, correlation analysis).
- **Dimensionality Reduction:** PCA (Principal Component Analysis) to remove redundancy.



K-NN Classifiers: Handling Interacting Attributes

Concept Overview

- **Feature interaction** occurs when the class label depends on a **combination of multiple attributes**, not on any single one.
- Individual features (like X or Y alone) may not separate the classes — the **joint relationship** matters.
- KNN can naturally handle such cases because it classifies based on **local neighborhood similarity** instead of fitting a global boundary.

Why Interaction Matters

- In many real datasets, attributes influence each other.
Example: The result (class) depends on both X and Y together.
- The class distribution may form **complex or alternating patterns** (like checkerboards or interleaved clusters).
- Linear classifiers (e.g., logistic regression) **cannot separate** such non-linear patterns effectively.

How KNN Handles Interacting Attributes

- **No assumption** about data distribution or decision boundary.
- Works in the **combined feature space (X, Y, ...)** to identify similar samples.
- **Local decisions** → classification based on the nearest neighbors in that region.
- Automatically adapts to **non-linear and irregular patterns** (e.g., checkerboard or multi-cluster data).



K-NN Classifiers: Handling Interacting Attributes

Go, change the world®

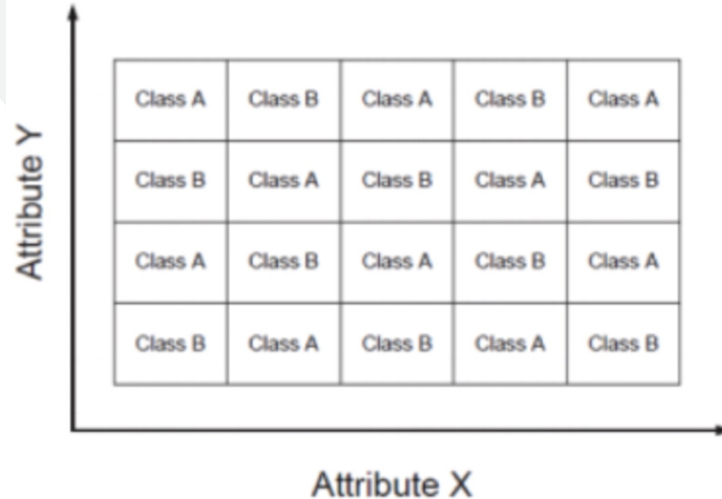
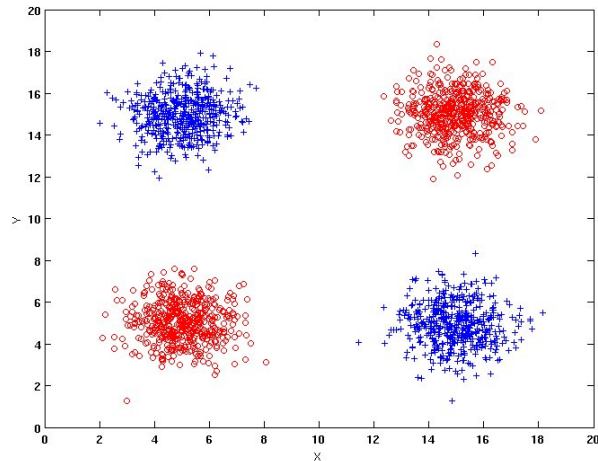
Illustrations

1.Cluster Pattern (First Figure):

1. Four clusters of two classes (A and B) — separated diagonally.
2. Each class appears in multiple regions → shows attribute interaction.

2.Checkerboard Pattern (Second Figure):

1. Alternating Class A and Class B blocks → class changes based on *both* X and Y values.
2. No linear boundary possible — but KNN classifies correctly by proximity.





Meaning of K

- **K** = Number of nearest neighbors used for classification/prediction.
- Analogy: “Let me check with my **K closest friends** before deciding.”
- The **choice of K** directly affects model performance.

K Value	Effect on Model
Small K	Model sensitive to noise → high variance
Large K	Model too smooth/generalized → high bias

2. Understanding Bias & Variance

•Bias:

- Oversimplifies the problem → misses patterns.
- *Example:* Student uses one formula for all math problems.

•Variance:

- Too sensitive to training data → overfits.
- *Example:* Student memorizes textbook questions, fails new ones.



3. Ideal Model

- Achieves **low bias** (captures patterns)
- And **low variance** (generalizes well)
- Balancing bias–variance is key to selecting the **optimal K**.

K value	Behavior	Impact on Performance	Analogy
Small K=1,3	Sensitive to noise	High variance, low bias	Relying on a single person's opinion- which could not be accurate
Moderate K=5,7,9	Captures local structure	Balanced trade-off	Asking a group of thoughtful people, thus getting good feedback
Large K=10,25,50	Includes irrelevant points	Low variance, high bias	Asking whole class with too many opinions- often not the one you were looking for



Scenario: Recommending study groups

If a student wants to join a particular study group based on interest and their availability.

K=1 joins a group with a single person, who might not be interested in that subject. Not a good choice.

K=3,5 joins a group with few people - might get good input.

K=20 joins a large group that may give a generic response not suiting your needs.

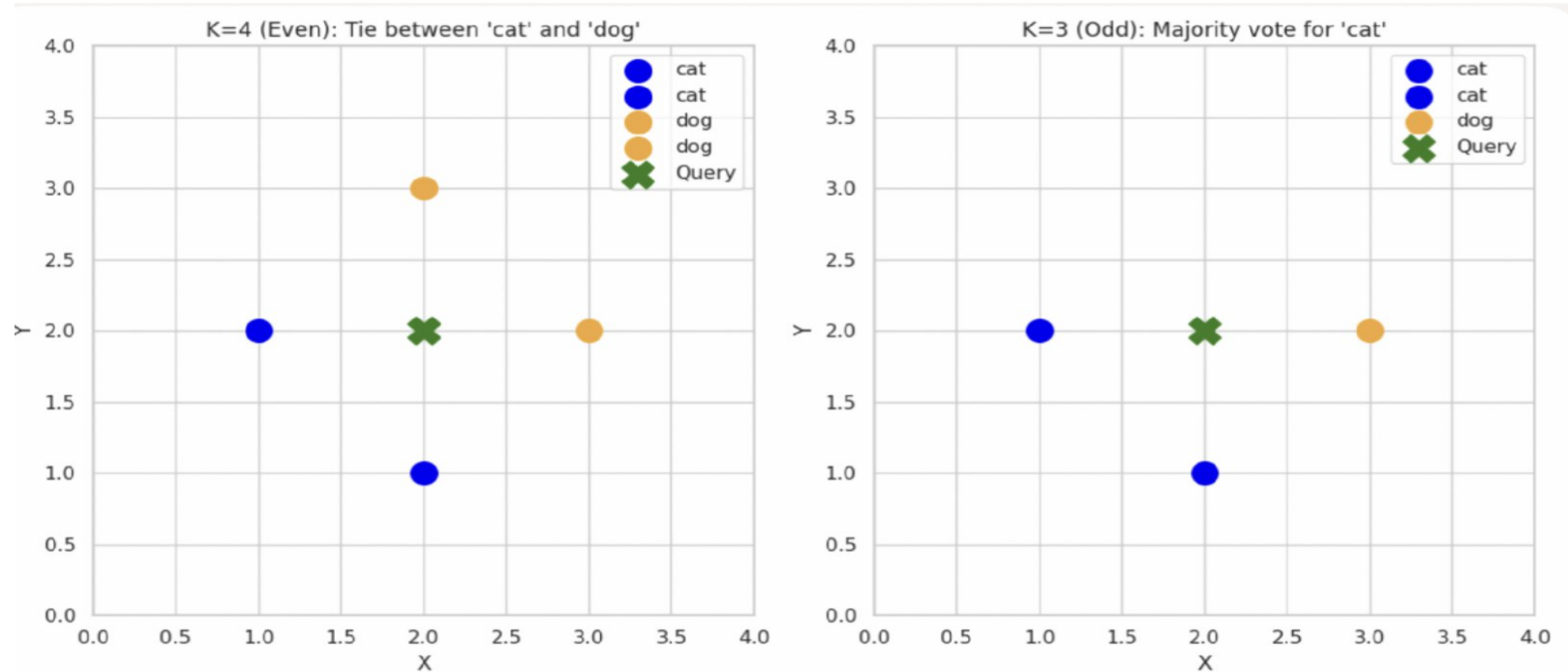
How to choose K:

- 1) Try different values of K to see which one gives the accurate result.
- 2) Choosing an odd number for K helps in avoiding ties as shown below.



Choice of K and It's Impact

Go, change the world®





KNN's biggest weakness:

It stores **all training data** and computes **distance with every sample** at prediction time → **slow for large datasets.**

Techniques to Improve Speed:

◆ A. Efficient Data Structures

1.k-d Trees:

- Divide data space into regions for faster neighbor search (best for low-dimension data).

2.Ball Trees / Metric Trees:

- Work better for medium to high dimensions.

3.Locality Sensitive Hashing (LSH):

- Groups nearby points into similar buckets using hash functions → used for high-dimensional or text data.

4.Approximate Similarity Search:

- Returns close-enough neighbors quickly — a good tradeoff between accuracy and speed.



◆ B. Data Reduction

1. Condensing:

- Keep a smaller subset of data that gives nearly the same accuracy.
- Example: Prototype selection.

2. Editing:

- Remove noisy or misclassified points → improves both accuracy and runtime.

Analogy:

“Instead of asking everyone in the city for advice (full dataset), you keep only the key representatives who think like the majority.”



RV College of
Engineering®

Naive Bayes Classifier: Basics of Probability Theory, Naive Bayes assumption.

Go, change the world®



- A probabilistic framework for solving classification problems
- Conditional Probability:
$$P(Y | X) = \frac{P(X, Y)}{P(X)}$$
- Bayes theorem:
$$P(X | Y) = \frac{P(X, Y)}{P(Y)}$$

$$P(Y | X) = \frac{P(X | Y)P(Y)}{P(X)}$$



Suppose you have invited two of your friends Alex and Martha to a dinner party. You know that Alex attends 40% of the parties he is invited to. Further, if Alex is going to a party, there is an 80% chance of Martha coming along. On the other hand, if Alex is not going to the party, the chance of Martha coming to the party is reduced to 30%. If Martha has responded that she will be coming to your party, what is the probability that Alex will also be coming?

1) Define the events

- $A = 1$: Alex goes to the party
- $A = 0$: Alex does not go
- $M = 1$: Martha goes to the party

2) Known probabilities

Prior probability of Alex going: $P(A = 1) = 0.4$

$P(A = 0) = 0.6$ (since total probability = 1)

Conditional probabilities of Martha going:

If Alex goes: $P(M = 1 | A = 1) = 0.8$

If Alex doesn't go: $P(M = 1 | A = 0) = 0.3$



3) Probability that Alex goes to the party provided Martha is going is: (applying Bayes Theorem)

$$P(A = 1 | M = 1) = \frac{P(M = 1 | A = 1) \cdot P(A = 1)}{P(M = 1)}$$

To compute $P(M=1)$ using law of total probability, we have:

$$P(M = 1) = P(M = 1 | A = 1) \cdot P(A = 1) + P(M = 1 | A = 0) \cdot P(A = 0)$$

Substituting values:

$$P(M=1) = (0.8 * 0.4) + (0.3 * 0.6) = 0.50$$

4) Substituting this into Bayes' formula:

$$P(A=1|M=1) = (0.8 * 0.4) / 0.5 = 0.64$$

If Martha is going to the party, there's a 64% chance Alex will go too—even though his original chance was only 40%.



- Posterior probability is the updated probability of a hypothesis or event after taking into account new evidence. It combines prior belief with observed data using Bayes' Theorem.
- In Bayesian statistics, we start with a **prior probability**—our initial belief about an event. When we observe new data, we update that belief to get the **posterior probability**.

$$P(Y | X) = \frac{P(X | Y)P(Y)}{P(X)}$$

- Where
- $P(Y|X)$ is known as Posterior probability of observing a class label Y for a data instance given its set of attribute values X
- $P(X|Y)$ is likelihood of observing X from the distribution of instances belonging to Y
- $P(Y)$ is the Prior Probability of Y before seeing data(independent of attribute values)
- $P(X)$ is the Evidence- total probability of observing X



<i>Tid</i>	Refund	Marital Status	Taxable Income	Evade
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Given a Test Record:

$X = (\text{Refund} = \text{No}, \text{Divorced}, \text{Income} = 120\text{K})$

- We need to estimate
 - $P(\text{Evade} = \text{Yes} \mid X)$ and $P(\text{Evade} = \text{No} \mid X)$
 - In the following we will replace
Evade = Yes by Yes, and Evade = No
by No



Given a Test Record:

$X = (\text{Refund} = \text{No}, \text{Divorced}, \text{Income} = 120\text{K})$

Tid	Refund	Marital Status	Taxable Income	Evade
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Using Bayes Theorem:

□ $P(\text{Yes} | X) = \frac{P(X | \text{Yes})P(\text{Yes})}{P(X)}$

□ $P(\text{No} | X) = \frac{P(X | \text{No})P(\text{No})}{P(X)}$

□ How to estimate $P(X | \text{Yes})$ and $P(X | \text{No})$?



Given a Test Record:

Tid	Refund	Marital Status	Taxable Income	Evade
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

$X = (\text{Refund} = \text{No}, \text{Divorced}, \text{Income} = 120\text{K})$

$P(X | \text{Yes}) =$

$P(\text{Refund} = \text{No} | \text{Yes}) \times$

$P(\text{Divorced} | \text{Yes}) \times$

$P(\text{Income} = 120\text{K} | \text{Yes})$

$P(X | \text{No}) =$

$P(\text{Refund} = \text{No} | \text{No}) \times$

$P(\text{Divorced} | \text{No}) \times$

$P(\text{Income} = 120\text{K} | \text{No})$



Estimate Probabilities from Data

Go, change the world®

Tid	Refund	Marital Status	Taxable Income	Evade
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

- $P(y)$ = fraction of instances of class y

- e.g., $P(\text{No}) = 7/10$,
 $P(\text{Yes}) = 3/10$

- For categorical attributes:

$$P(X_i = c | y) = n_c / n$$

- where $|X_i = c|$ is number of instances having attribute value $X_i = c$ and belonging to class y
 - Examples:

$$P(\text{Status}=\text{Married}|\text{No}) = 4/7$$

$$P(\text{Refund}=\text{Yes}|\text{Yes})=0$$



- For continuous attributes:
 - **Discretization:** Partition the range into bins:
 - ◆ Replace continuous value with bin value
 - Attribute changed from continuous to ordinal
 - **Probability density estimation:**
 - ◆ Assume attribute follows a normal distribution
 - ◆ Use data to estimate parameters of distribution (e.g., mean and standard deviation)
 - ◆ Once probability distribution is known, use it to estimate the conditional probability $P(X_i|Y)$

Tid	Refund	Marital Status	Taxable Income	Evade
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

- Normal distribution:

$$P(X_i | Y_j) = \frac{1}{\sqrt{2\pi\sigma_{ij}^2}} e^{-\frac{(X_i - \mu_{ij})^2}{2\sigma_{ij}^2}}$$
- One for each (X_i, Y_i) pair

- For (Income, Class=No):
 - If Class=No
 - ◆ sample mean = 110
 - ◆ sample variance = 2975

$$P(\text{Income} = 120 | \text{No}) = \frac{1}{\sqrt{2\pi (54.54)}} e^{-\frac{(120-110)^2}{2(2975)}} = 0.0072$$



Given a Test Record:

Naïve Bayes Classifier:

$$P(\text{Refund} = \text{Yes} \mid \text{No}) = 3/7$$

$$P(\text{Refund} = \text{No} \mid \text{No}) = 4/7$$

$$P(\text{Refund} = \text{Yes} \mid \text{Yes}) = 0$$

$$P(\text{Refund} = \text{No} \mid \text{Yes}) = 1$$

$$P(\text{Marital Status} = \text{Single} \mid \text{No}) = 2/7$$

$$P(\text{Marital Status} = \text{Divorced} \mid \text{No}) = 1/7$$

$$P(\text{Marital Status} = \text{Married} \mid \text{No}) = 4/7$$

$$P(\text{Marital Status} = \text{Single} \mid \text{Yes}) = 2/3$$

$$P(\text{Marital Status} = \text{Divorced} \mid \text{Yes}) = 1/3$$

$$P(\text{Marital Status} = \text{Married} \mid \text{Yes}) = 0$$

For Taxable Income:

If class = No: sample mean = 110

sample variance = 2975

If class = Yes: sample mean = 90

sample variance = 25

$$X = (\text{Refund} = \text{No}, \text{Divorced}, \text{Income} = 120\text{K})$$

- $$\begin{aligned} P(X \mid \text{No}) &= P(\text{Refund}=\text{No} \mid \text{No}) \\ &\times P(\text{Divorced} \mid \text{No}) \\ &\times P(\text{Income}=120\text{K} \mid \text{No}) \\ &= 4/7 \times 1/7 \times 0.0072 = 0.0006 \end{aligned}$$
- $$\begin{aligned} P(X \mid \text{Yes}) &= P(\text{Refund}=\text{No} \mid \text{Yes}) \\ &\times P(\text{Divorced} \mid \text{Yes}) \\ &\times P(\text{Income}=120\text{K} \mid \text{Yes}) \\ &= 1 \times 1/3 \times 1.2 \times 10^{-9} = 4 \times 10^{-10} \end{aligned}$$

Since $P(X|\text{No})P(\text{No}) > P(X|\text{Yes})P(\text{Yes})$

Therefore $P(\text{No}|X) > P(\text{Yes}|X)$

=> Class = No



Naïve Bayes Classifier can make decisions on partial info

about attributes in Test Record

Go, change the world®

Even in absence of information about any attributes, we can use Apriori Probabilities of Class Variable:

Naïve Bayes Classifier:

$$P(\text{Refund} = \text{Yes} \mid \text{No}) = 3/7 \quad P(\text{Refund} = \text{No} \mid \text{No}) = 4/7$$

$$P(\text{Refund} = \text{Yes} \mid \text{Yes}) = 0$$

$$P(\text{Refund} = \text{No} \mid \text{Yes}) = 1$$

$$P(\text{Marital Status} = \text{Single} \mid \text{No}) = 2/7$$

$$P(\text{Marital Status} = \text{Divorced} \mid \text{No}) = 1/7$$

$$P(\text{Marital Status} = \text{Married} \mid \text{No}) = 4/7$$

$$P(\text{Marital Status} = \text{Single} \mid \text{Yes}) = 2/3$$

$$P(\text{Marital Status} = \text{Divorced} \mid \text{Yes}) = 1/3$$

$$P(\text{Marital Status} = \text{Married} \mid \text{Yes}) = 0$$

For Taxable Income:

If class = No: sample mean = 110

sample variance = 2975

If class = Yes: sample mean = 90

sample variance = 25

If we only know that marital status is Divorced, then: $P(\text{Yes} \mid$

$$\text{Divorced}) = 1/3 \times 3/10 / P(\text{Divorced})$$

$$P(\text{No} \mid \text{Divorced}) = 1/7 \times 7/10 / P(\text{Divorced})$$

$$P(\text{Yes}) = 3/10$$

$$P(\text{No}) = 7/10$$

If we also know that Refund = No, then

$$P(\text{Yes} \mid \text{Refund} = \text{No}, \text{Divorced}) = 1 \times 1/3 \times 3/10 / P(\text{Divorced}, \text{Refund} = \text{No})$$

$$P(\text{No} \mid \text{Refund} = \text{No}, \text{Divorced}) = 4/7 \times 1/7 \times 7/10 / P(\text{Divorced}, \text{Refund} = \text{No})$$

If we also know that Taxable Income = 120, then

$$P(\text{Yes} \mid \text{Refund} = \text{No}, \text{Divorced}, \text{Income} = 120) = 1.2 \times 10^{-9} \times 1 \times 1/3 \times 3/10 / P(\text{Divorced}, \text{Refund} = \text{No}, \text{Income} = 120)$$

$$P(\text{No} \mid \text{Refund} = \text{No}, \text{Divorced}, \text{Income} = 120) = 0.0072 \times 4/7 \times 1/7 \times 7/10 /$$

$$P(\text{Divorced}, \text{Refund} = \text{No}, \text{Income} = 120)$$



Given a Test Record:

X = (Married)

Naïve Bayes Classifier:

$$P(\text{Refund} = \text{Yes} \mid \text{No}) = 3/7$$

$$P(\text{Refund} = \text{No} \mid \text{No}) = 4/7$$

$$P(\text{Refund} = \text{Yes} \mid \text{Yes}) = 0$$

$$P(\text{Refund} = \text{No} \mid \text{Yes}) = 1$$

$$P(\text{Marital Status} = \text{Single} \mid \text{No}) = 2/7$$

$$P(\text{Marital Status} = \text{Divorced} \mid \text{No}) = 1/7$$

$$P(\text{Marital Status} = \text{Married} \mid \text{No}) = 4/7$$

$$P(\text{Marital Status} = \text{Single} \mid \text{Yes}) = 2/3$$

$$P(\text{Marital Status} = \text{Divorced} \mid \text{Yes}) = 1/3$$

$$P(\text{Marital Status} = \text{Married} \mid \text{Yes}) = 0$$

$$P(\text{Yes}) = 3/10$$

$$P(\text{No}) = 7/10$$

$$P(\text{Yes} \mid \text{Married}) = 0 \times 3/10 / P(\text{Married})$$

$$P(\text{No} \mid \text{Married}) = 4/7 \times 7/10 / P(\text{Married})$$

For Taxable Income:

If class = No: sample mean = 110

sample variance = 2975

If class = Yes: sample mean = 90

sample variance = 25



Consider the table with Tid = 7 deleted

Tid	Refund	Marital Status	Taxable Income	Evade
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Given $X = (\text{Refund} = \text{Yes}, \text{Divorced}, 120\text{K})$

$$P(X | \text{No}) = \frac{2}{6} \times 0 \times 0.0083 = 0$$

$$P(X | \text{Yes}) = 0 \times \frac{1}{3} \times 1.2 \times 10^{-9} = 0$$

Naïve Bayes Classifier:

$$P(\text{Refund} = \text{Yes} | \text{No}) = \frac{2}{6}$$

$$P(\text{Refund} = \text{No} | \text{No}) = \frac{4}{6}$$

$$P(\text{Refund} = \text{Yes} | \text{Yes}) = 0$$

$$P(\text{Refund} = \text{No} | \text{Yes}) = \frac{1}{3}$$

$$P(\text{Marital Status} = \text{Single} | \text{No}) = \frac{2}{6}$$

$$P(\text{Marital Status} = \text{Divorced} | \text{No}) = 0$$

$$P(\text{Marital Status} = \text{Married} | \text{No}) = \frac{4}{6}$$

$$P(\text{Marital Status} = \text{Single} | \text{Yes}) = \frac{2}{3}$$

$$P(\text{Marital Status} = \text{Divorced} | \text{Yes}) = \frac{1}{3}$$

$$P(\text{Marital Status} = \text{Married} | \text{Yes}) = \frac{0}{3}$$

For Taxable Income:

If class = No: sample mean = 91

sample variance = 685

If class = No: sample mean = 90

sample variance = 25

**Naïve Bayes will not be able to classify X
as Yes or No!**



- If one of the conditional probabilities is zero, then the entire expression becomes zero
- Need to use other estimates of conditional probabilities than simple fractions

- **Probability estimation:**

original: $P(X_i = c|y) = \frac{n_c}{n}$

Laplace Estimate: $P(X_i = c|y) = \frac{n_c + 1}{n + v}$

m – estimate: $P(X_i = c|y) = \frac{n_c + mp}{n + m}$

n : number of training
instances belonging to class y

n_c : number of instances with
 $X_i = c$ and $Y = y$

v : total number of attribute
values that X_i can take

p : initial estimate of
($P(X_i = c|y)$ known apriori

m : hyper-parameter for our
confidence in p



Example of Naïve Bayes Classifier

Go, change the world®

Name	Give Birth	Can Fly	Live in Water	Have Legs	Class
human	yes	no	no	yes	mammals
python	no	no	no	no	non-mammals
salmon	no	no	yes	no	non-mammals
whale	yes	no	yes	no	mammals
frog	no	no	sometimes	yes	non-mammals
komodo	no	no	no	yes	non-mammals
bat	yes	yes	no	yes	mammals
pigeon	no	yes	no	yes	non-mammals
cat	yes	no	no	yes	mammals
leopard shark	yes	no	yes	no	non-mammals
turtle	no	no	sometimes	yes	non-mammals
penguin	no	no	sometimes	yes	non-mammals
porcupine	yes	no	no	yes	mammals
eel	no	no	yes	no	non-mammals
salamander	no	no	sometimes	yes	non-mammals
gila monster	no	no	no	yes	non-mammals
platypus	no	no	no	yes	mammals
owl	no	yes	no	yes	non-mammals
dolphin	yes	no	yes	no	mammals
eagle	no	yes	no	yes	non-mammals

A: attributes

M: mammals

N: non-mammals

Give Birth	Can Fly	Live in Water	Have Legs	Class
yes	no	yes	no	?



Example of Naïve Bayes Classifier

Go, change the world®

Name	Give Birth	Can Fly	Live in Water	Have Legs	Class
human	yes	no	no	yes	mammals
python	no	no	no	no	non-mammals
salmon	no	no	yes	no	non-mammals
whale	yes	no	yes	no	mammals
frog	no	no	sometimes	yes	non-mammals
komodo	no	no	no	yes	non-mammals
bat	yes	yes	no	yes	mammals
pigeon	no	yes	no	yes	non-mammals
cat	yes	no	no	yes	mammals
leopard shark	yes	no	yes	no	non-mammals
turtle	no	no	sometimes	yes	non-mammals
penguin	no	no	sometimes	yes	non-mammals
porcupine	yes	no	no	yes	mammals
eel	no	no	yes	no	non-mammals
salamander	no	no	sometimes	yes	non-mammals
gila monster	no	no	no	yes	non-mammals
platypus	no	no	no	yes	mammals
owl	no	yes	no	yes	non-mammals
dolphin	yes	no	yes	no	mammals
eagle	no	yes	no	yes	non-mammals

Give Birth	Can Fly	Live in Water	Have Legs	Class
yes	no	yes	no	?

A: attributes

M: mammals

N: non-mammals

$$P(A | M) = \frac{6}{7} \times \frac{6}{7} \times \frac{2}{7} \times \frac{2}{7} = 0.06$$

$$P(A | N) = \frac{1}{13} \times \frac{10}{13} \times \frac{3}{13} \times \frac{4}{13} = 0.0042$$

$$P(A | M)P(M) = 0.06 \times \frac{7}{20} = 0.021$$

$$P(A | N)P(N) = 0.004 \times \frac{13}{20} = 0.0027$$

P(A|M)P(M) > P(A|N)P(N)

=> Mammals



Example of Naïve Bayes Classifier

Go, change the world®

Consider the data set shown in the table:

Instance	A	B	C	Class
1	0	0	0	+
2	0	0	1	-
3	0	1	1	-
4	0	1	1	-
5	0	0	1	+
6	1	0	1	+
7	1	0	1	-
8	1	0	1	-
9	1	1	1	+
10	1	0	1	+

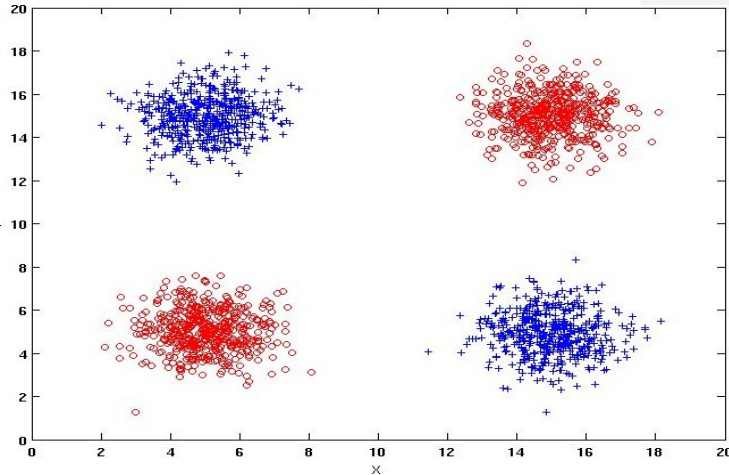
- 1) Estimate the conditional probabilities for $P(A|+)$, $P(B|+)$, $P(C|+)$, $P(A|-)$, $P(B|-)$ and $P(C|-)$.
- 2) Use the estimate of conditional probabilities given in the previous question to predict the class label for a test sample ($A=0$, $B=1$, $C=0$) using the naïve Bayes approach.



- **Robust to isolated noise points-** uses probabilistic averaging, a few noisy or mislabeled data points don't drastically affect the overall probability estimates.
- **Handle missing values by ignoring the instance during probability estimate calculations.** Example: If income is missing, it still uses refund and marital status to make a decision.
- **Robust to irrelevant attributes-** tend to have uniform conditional probabilities as the feature contributes equally to all class probabilities, so it doesn't distort or bias the final prediction.
- **Redundant and correlated attributes will violate class conditional assumption**
 - Use other techniques such as Bayesian Belief Networks (BBN)



- How does Naïve Bayes perform on the following dataset?



- 1) Assumes Feature Independence- **X and Y are jointly informative**—the class depends on their combination, not on either alone.
- 2) Uses Axis-Aligned Decision Boundaries- builds decision boundaries that are **parallel to the axes**, which can't capture diagonal or quadrant-based separations.
- 3) Overlapping Class Distributions- Naive Bayes might average across these, leading to **confused predictions** near cluster centers.

Conditional independence of attributes is violated



- What Is Conditional Independence?

Naive Bayes assumes that:

Given the class label, all features are independent of each other.

Formally, for features $X_1, X_2, \dots, X_n$ and class C :

$$P(X_1, X_2, \dots, X_n | C) = P(X_1 | C) \cdot P(X_2 | C) \cdots P(X_n | C)$$

Example: Suppose you're predicting tax evasion:

Feature 1: **Job Title**

Feature 2: **Income**

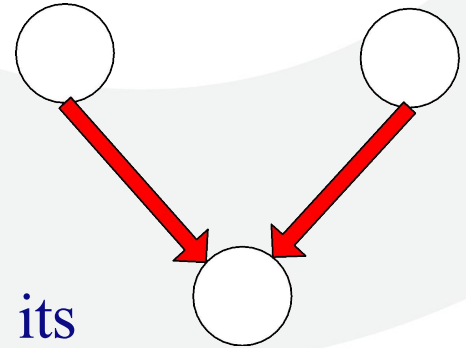
Even if you know someone's class (Evade = Yes or No), **Job Title and Income are still related**—CEOs tend to have high income. So:

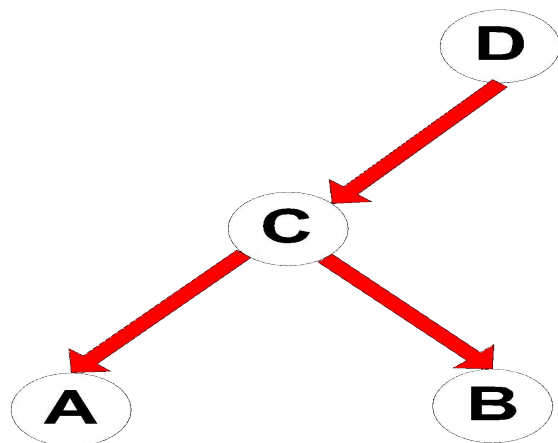
$P(\text{Income} | \text{Evade})$ and $P(\text{Job Title} | \text{Evade})$ are not independent

Naive Bayes multiplies both, **as if they're separate pieces of evidence**, which inflates the prediction. But this **overstates the evidence**, because both features are saying the same thing. It's like **counting the same clue twice**.



- Provides graphical representation of probabilistic relationships among a set of random variables
- Consists of:
 - A directed acyclic graph (dag)
 - ◆ Node corresponds to a variable
 - ◆ Arc corresponds to dependence relationship between a pair of variables
 - A probability table associating each node to its immediate parent





- C's children: A, B
- C's parent: D
- C's descendants: A, B (and any nodes reachable from A or B)
- C's non-descendants: D (and any nodes not downstream of C)

C is conditionally independent of its non-descendants (like D), given its parents.

But since D is a parent of C, this rule doesn't apply to D directly. Instead, it applies to other non-descendants (if any existed) once D is known.

- A node in a Bayesian network is conditionally independent of all of its nondescendants, if its parents are known



- If X does not have any parents, table contains prior probability $P(X)$
- If X has only one parent (Y), table contains conditional probability $P(X|Y)$
- If X has multiple parents ($Y_1, Y_2, \dots, Y_k$), table contains conditional probability $P(X|Y_1, Y_2, \dots, Y_k)$

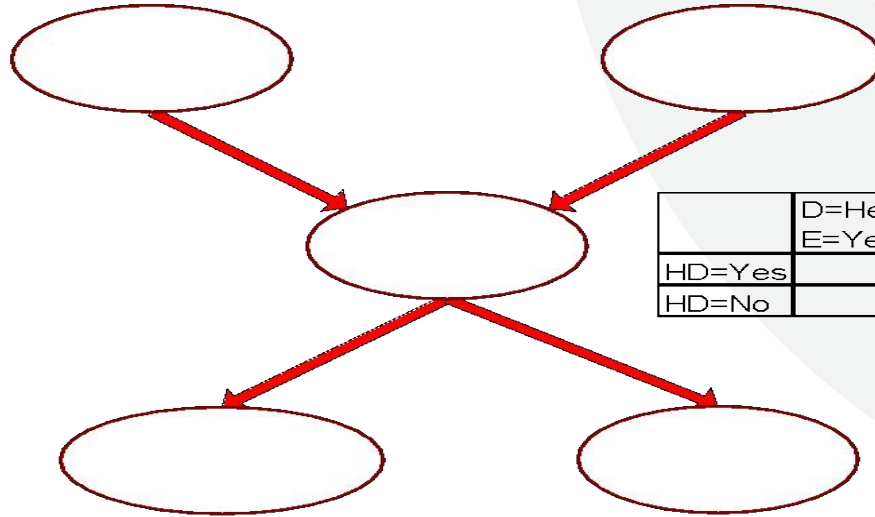


Example of Bayesian Belief Networks

Go, change the world®

Exercise=Yes	0.7
Exercise=No	0.3

Diet=Healthy	0.25
Diet=Unhealthy	0.75



	D=Healthy E=Yes	D=Healthy E=No	D=Unhealthy E=Yes	D=Unhealthy E=No
HD=Yes	0.25	0.45	0.55	0.75
HD=No	0.75	0.55	0.45	0.25

	HD=Yes	HD=No
CP=Yes	0.8	0.01
CP=No	0.2	0.99

	HD=Yes	HD=No
BP=High	0.85	0.2
BP=Low	0.15	0.8



- Given: $X = (E=\text{No}, D=\text{Yes}, CP=\text{Yes}, BP=\text{High})$
 - Compute $P(HD|E,D,CP,BP)$?
- $P(HD=\text{Yes} | E=\text{No}, D=\text{Yes}) = 0.45$ $P(CP=\text{Yes} | HD=\text{Yes}) = 0.8$ $P(BP=\text{High} | HD=\text{Yes}) = 0.85$
 - $P(HD=\text{Yes} | E=\text{No}, D=\text{Yes}, CP=\text{Yes}, BP=\text{High})$
 $\propto 0.45 \times 0.8 \times 0.85 = 0.306$
- $P(HD=\text{No} | E=\text{No}, D=\text{Yes}) = 0.45$ $P(CP=\text{Yes} | HD=\text{No}) = 0.01$ $P(BP=\text{High} | HD=\text{No}) = 0.2$
 - $P(HD=\text{No} | E=\text{No}, D=\text{Yes}, CP=\text{Yes}, BP=\text{High})$
 $\propto 0.45 \times 0.01 \times 0.2 = 0.0009$

**Classify X
as Yes**



RV College of
Engineering®

Logistic Regression: Logistic Regression as a Generalized Linear Model, Learning Model Parameters, Characteristics of Logistic Regression

Go, change the world®



Generative Models (e.g., Naive Bayes)

These models try to **learn how the data is generated** for each class.

They estimate:

$P(x | y)$: the probability of seeing features x given class y

$P(y)$: the prior probability of each class

Then they use **Bayes' theorem** to compute:

$$P(y | x) = \frac{P(x | y) \cdot P(y)}{P(x)}$$

Useful when you want to **simulate or understand** the data distribution.

Discriminative Models (e.g., Logistic Regression)

These models **skip modeling how data is generated**.

They directly estimate:

$$P(y | x)$$

— the probability of a class given the observed features.

They focus on **drawing boundaries** between classes rather than modeling the full data distribution.

Useful when your goal is **classification accuracy**, not data generation.



Goal of Classification

We want to predict the class label $y \in \{0,1\}$ for a given data instance x .

Instead of modeling the data generation process $P(x | y)$ (as generative models do, logistic regression directly models the posterior probability $P(y | x)$ —this is the **discriminative approach**.

Odds and Linear Predictor

The **odds** of class 1 vs class 0 is given by:

$$\frac{P(y = 1 | x)}{P(y = 0 | x)} = e^z = e^{w^T x + b}$$

This ratio is known as the odds. If this ratio is greater than 1, then x is classified as $y=1$. Otherwise, it is assigned to class $y=0$.

From Odds to Probability

Since $P(y = 0 | x) + P(y = 1 | x) = 1$, we can express the posterior probability as:

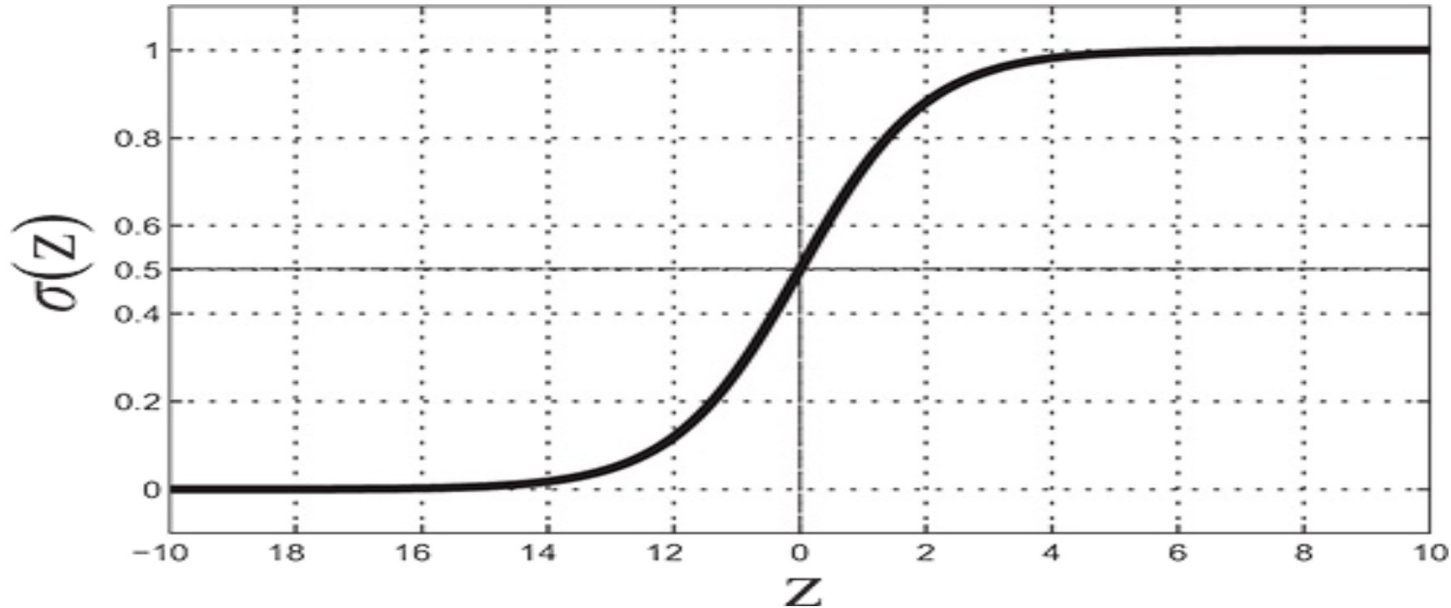
$$P(y = 1 | x) = \frac{1}{1 + e^{-z}} = \sigma(z)$$

This is the **sigmoid function**, denoted $\sigma(z)$



An Example Curve

Go, change the world®



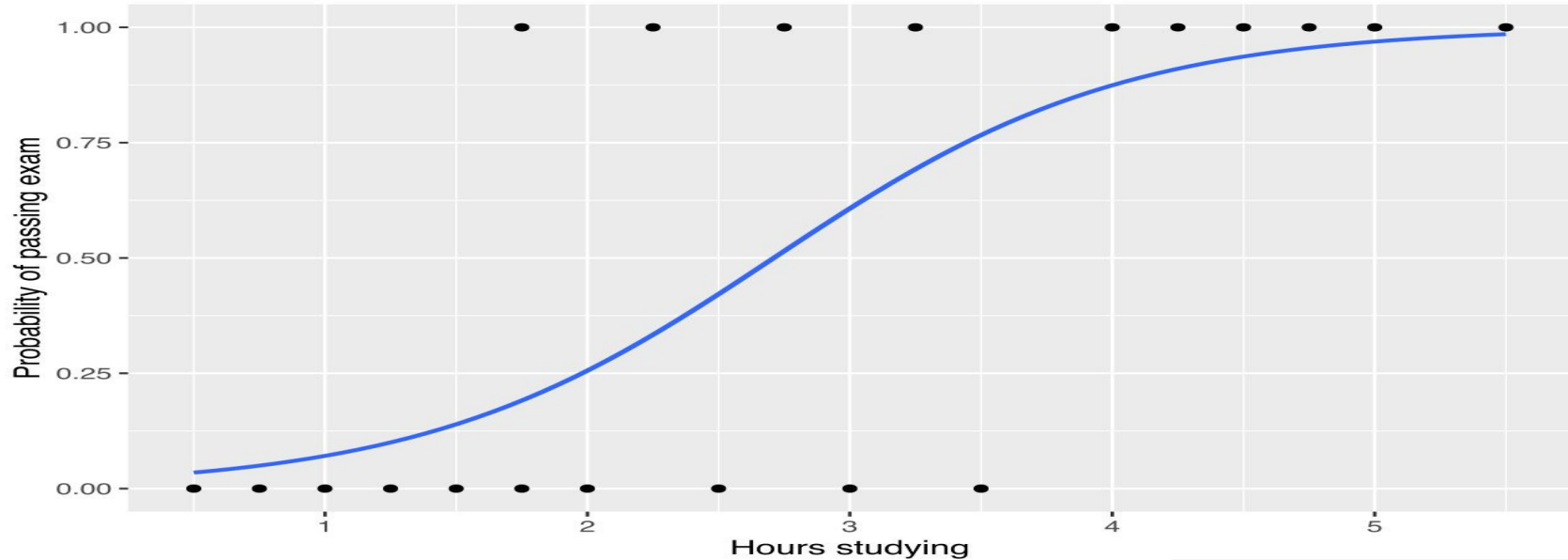
- If $\sigma(z) > 0.5$ (i.e., $z > 0$), predict **class 1**.
- If $\sigma(z) < 0.5$ (i.e., $z < 0$), predict **class 0**.



An Example Curve

Go, change the world®

Probability of passing exam versus hours of studying



Example graph of a logistic regression curve fitted to data. The curve shows the probability of passing an exam (binary dependent variable) versus hours studying (scalar independent variable).



Logistic Regression as a Generalized Linear Model

Go, change the world®

- Logistic regression is a type of generalized linear model (GLM) because it models the log-odds of a binary outcome as a linear function of the input variables, using a link function to relate the linear predictor to the probability.

$$g(\mu) = z = \mathbf{w}^T \mathbf{x} + b.$$

$$g(\mu) = \log \left(\frac{\mu}{1 - \mu} \right).$$

- Even though logistic regression has relationships with regression models, it is a classification model since the computed posterior probabilities are eventually used to determine the class label of a data instance.



- The parameters of logistic regression, (w, b) , are estimated during training using a statistical approach known as the maximum likelihood estimation (MLE) method.

$$\mathcal{L}(\mathbf{w}, b) = \prod_{i=1}^n P(y_i | \mathbf{x}_i, \mathbf{w}, b) = \prod_{i=1}^n P(y = 1 | \mathbf{x}_i)^{y_i} \times P(y = 0 | \mathbf{x}_i)^{1-y_i}.$$



$\mathbf{x}_i$: Feature vector for the i -th training instance

y_i : True label (0 or 1) for the i -th instance

$P(y = 1 \mid \mathbf{x}_i)$: Predicted probability of class 1 for instance i

$P(y=0 \mid \mathbf{x}_i) = 1 - P(y=1 \mid \mathbf{x}_i)$

$\mathbf{w}, b$: Parameters of the logistic regression model

The product over all instances gives the **overall likelihood** of observing the training data given the model parameters.

- The goal of MLE is to **find the parameters** $\mathbf{w}, b$ that **maximize** this likelihood.
- we take the **log** of this function to simplify computation:

$$\log \mathcal{L}(\mathbf{w}, b) = \sum_{i=1}^n [y_i \log P(y = 1 \mid \mathbf{x}_i) + (1 - y_i) \log P(y = 0 \mid \mathbf{x}_i)]$$

This is called the **log-likelihood**, and we optimize it using gradient descent.



- ❑ Discriminative model for classification
- ❑ Decision Boundary: Learns only linear decision boundaries (limited expressive power).
- ❑ Multiclass Extension: Easily extended to multiclass problems via multinomial (softmax) regression.
- ❑ Interpretability: Parameters (weights) are directly interpretable — each weight shows the importance and direction of influence of the corresponding feature on the class label.
- ❑ High-Dimensional Robustness: Performs better than distance-based methods (e.g., k-NN) in high dimensions because it does not compute densities or distances in the feature space.
- ❑ Feature Handling: Irrelevant features → learns near-zero weights.
Interacting features → handles naturally because all weights are learned jointly.
Redundant/duplicate features → assigns similar weights without hurting performance.
Many irrelevant/redundant features in high dimensions → can still lead to overfitting.
- ❑ Missing Values: Cannot naturally handle missing values: Training: instances with missing values are typically discarded.
Testing: prediction fails if any feature is missing (since prediction requires a complete weighted sum).



RV College of
Engineering®

Ensemble Techniques

Go, change the world®



- There are techniques for improving classification accuracy by aggregating the predictions of multiple classifiers.
 - known as *ensemble or classifier combination* methods.
- An ensemble method constructs a set of **base classifiers** from training data and performs classification
 - by taking a vote on the predictions made by each base classifier.



- Construct a set of base classifiers learned from the training data
- Predict class label of test records by combining the predictions made by multiple classifiers (e.g., by taking majority vote)



- Construct a set of base classifiers learned from the training data
- Predict class label of test records by combining the predictions made by multiple classifiers (e.g., by taking majority vote)



Example: Why do Ensemble Methods Work?

Go, change the world®

- Suppose there are 25 base classifiers
 - Each classifier has error rate, $\epsilon = 0.35$
 - Majority vote of classifiers used for classification
 - If all classifiers are identical:
 - ◆ Error rate of ensemble = ϵ (0.35)
 - If all classifiers are independent (errors are uncorrelated):
 - ◆ Error rate of ensemble = probability of having more than half of base classifiers being wrong



Example: Why do Ensemble Methods Work?

Go, change the world®

$$e_{\text{ensemble}} = \sum_{i=13}^{25} \binom{25}{i} \epsilon^i (1 - \epsilon)^{25-i} = 0.06$$

Each classifier has $p = 0.35$ chance of being wrong $\rightarrow p = 0.65$ chance of being correct.
Majority voting (13 or more out of 25 must be correct for the ensemble to be correct).

Number of classifiers

Individual error

Ensemble error (majority vote)

25

35%

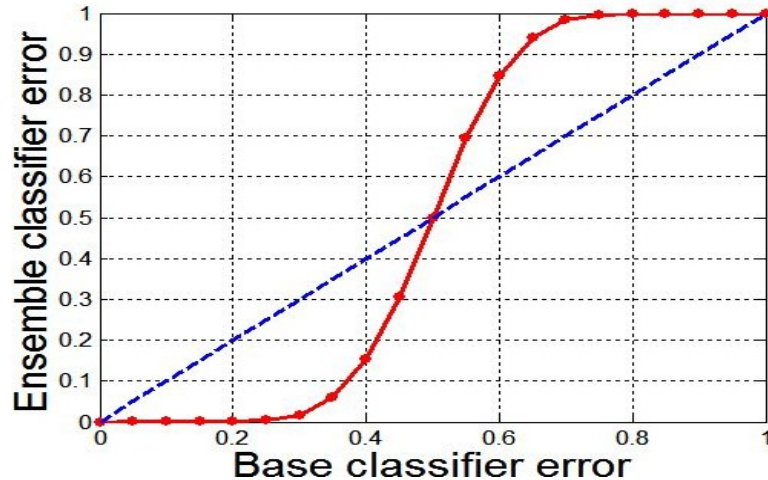
~0.078% (almost 450 times
better)



Necessary Conditions for Ensemble Methods

Go, change the world®

- Ensemble Methods work better than a single base classifier if:
 1. All base classifiers are independent of each other
 2. All base classifiers perform better than random guessing (error rate < 0.5 for binary classification)



Classification error for an ensemble of 25 base classifiers, assuming their errors are uncorrelated.

Comparison between errors of base classifiers and errors of the ensemble classifier



Necessary Conditions for Ensemble Methods

Imagine you have 25 classifiers and a test example (a picture of a cat).

Scenario : What happens on this cat image. Are the classifiers “independent”?

Perfectly identical / zero independence All 25 classifiers misclassify it as “dog” (because they are copies of each other or learned exactly the same patterns) Errors are 100% correlated
→ no independence

Very low independence 24 of them misclassify it as “dog”, only 1 gets it right (they all learned almost the same thing, maybe from very similar training subsets) Errors highly correlated

Moderate independence About 10 misclassify it as “dog”, the other 15 correctly say “cat” (some classifiers were trained on different subsets, different features, or different algorithms) Errors somewhat uncorrelated

High (near-perfect) independence Whether one classifier mistakes this cat for a dog tells you almost nothing about what the others will do. Some get it right, some wrong, almost randomly with respect to each other Errors nearly independent → ideal for ensembles



Necessary Conditions for Ensemble Methods

Go, change the world®

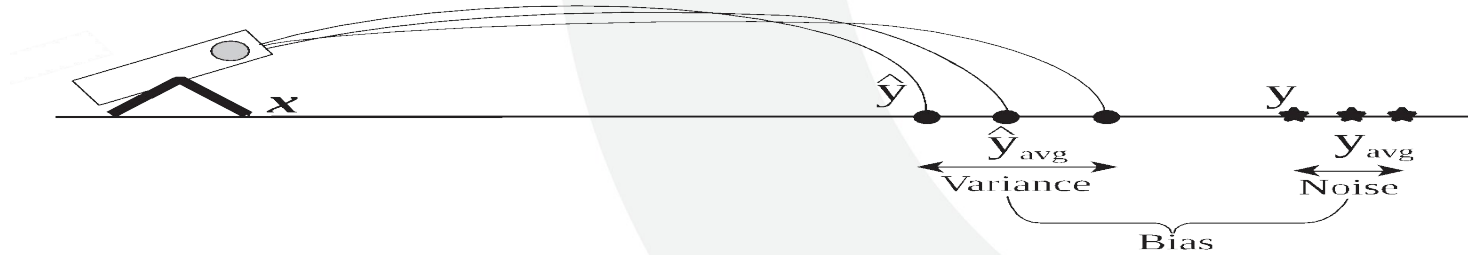
Classifier performance	Accuracy	Error rate	Is it better than random guessing?	Can it safely go into an ensemble?
Gets 45% correct	45%	0.55	No	Dangerous – will hurt the ensemble
Gets 50% correct	50%	0.50	No (equal to random)	Gives no benefit
Gets 51% correct	51%	0.49	Yes (slightly better)	OK – small help
Gets 65% correct	65%	0.35	Yes	Good base learner
Gets 80% correct	80%	0.20	Yes	Very strong base learner



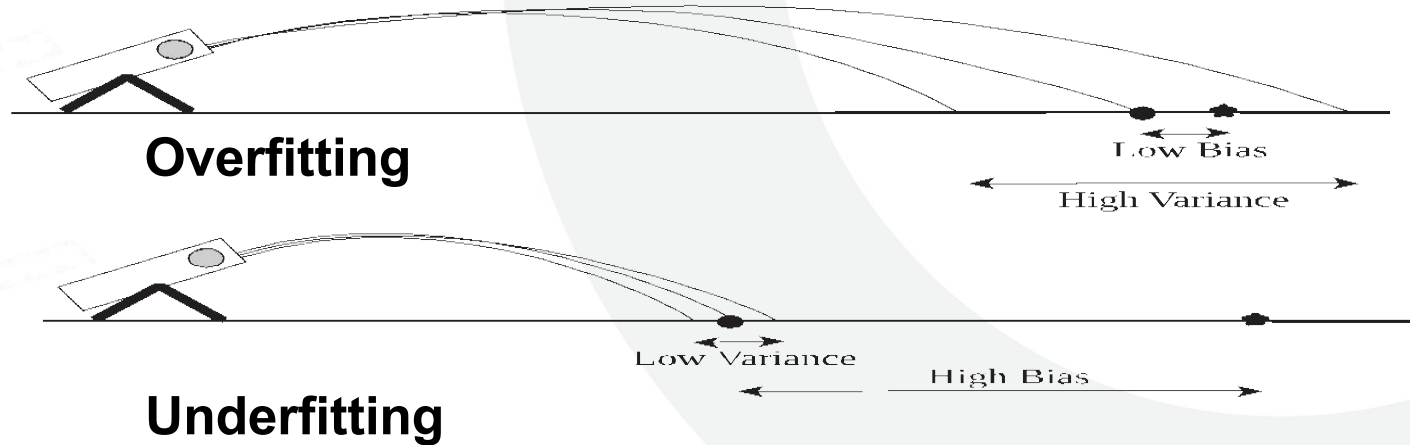
- Ensemble Methods work best with **unstable base classifiers**
 - Classifiers that are sensitive to minor perturbations in training set, due to *high model complexity*
 - Examples: Unpruned decision trees, ANNs, ...

Bias-Variance Decomposition

Go, change the world®



Component	Visual Cue in Image	Meaning
Bias	Distance between $\hat{y}_{avg}$ and y	Systematic error — how far the average prediction is from the true value
Variance	Spread of $\hat{y}$ around $\hat{y}_{avg}$	Sensitivity to training data — how predictions vary across models
Noise	Spread of y around y_{avg}	Irreducible error due to randomness in data

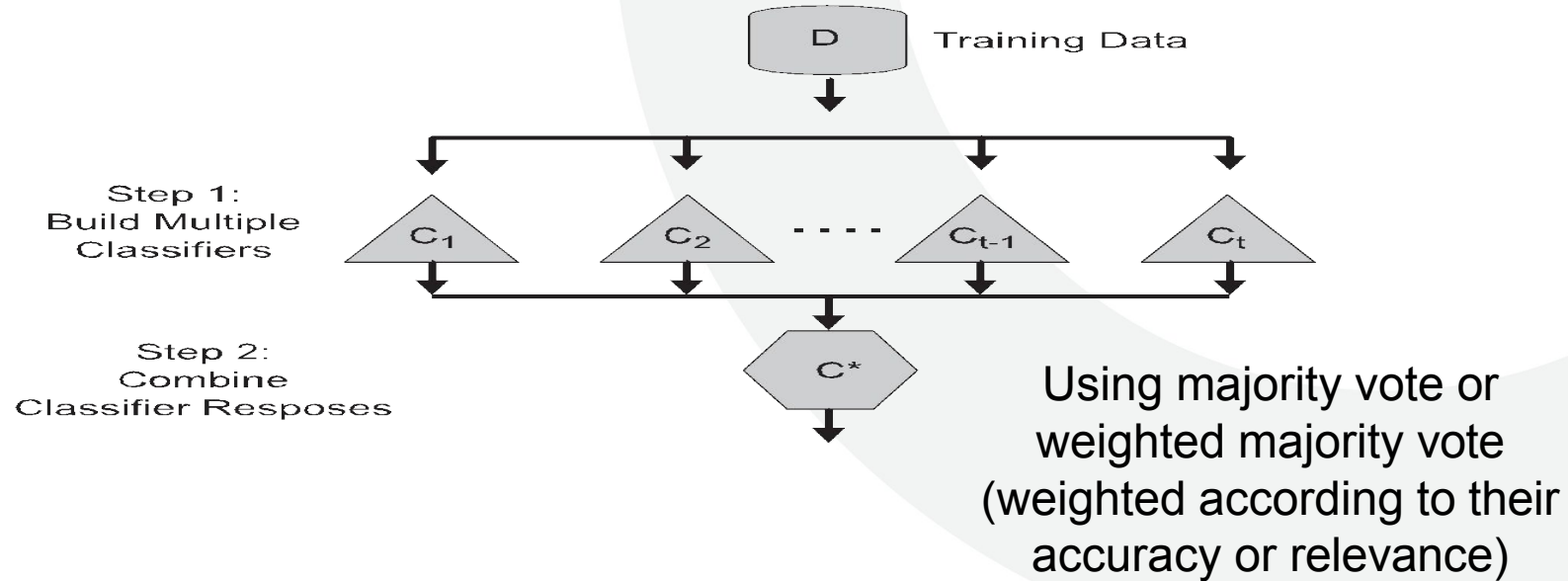


Ensemble methods reduce variance by training many models on different variations of the data and then combining (aggregating) their predictions.



General Approach For Ensemble Learning

Go, change the world®



A logical view of the ensemble learning method.



- By manipulating training set
 - Example: bagging, boosting
- By manipulating input features
 - Example: random forests
- By manipulating class labels
 - Example: error-correcting output coding
- By manipulating learning algorithm
 - Example: injecting randomness in the initial weights of ANN



- By manipulating training set
 - multiple training sets are created by resampling the original data according to some sampling distribution and constructing a classifier from each training set.
 - The sampling distribution determines how likely it is that an example will be selected for training, and it may vary from one trial to another.
 - Example: bagging, boosting
- By manipulating input features
 - a subset of input features is chosen to form each training set. The subset can be either chosen randomly or based on the recommendation of domain experts.
 - this approach works very well with data sets that contain highly redundant features
 - Example: random forests --ensemble method that manipulates its input features and uses decision trees as its base classifiers



- By manipulating class labels
 - Example: error-correcting output coding
 - can be used when the number of classes is sufficiently large.
 - The training data is transformed into a binary class problem by randomly partitioning the class labels into two disjoint subsets, A_0 and A_1 .
 - Training examples whose class label belongs to the subset A_0 are assigned to class 0, while those that belong to the subset A_1 are assigned to class 1.
 - The relabeled examples are then used to train a base classifier.
 - By repeating this process multiple times, an ensemble of base classifiers is obtained. When a test example is presented, each base classifier C_i is used to predict its class label.
 - If the test example is predicted as class 0, then all the classes that belong to A_0 will receive a vote. Conversely, if it is predicted to be class 1, then all the classes that belong to A_1 will receive a vote.
 - The votes are tallied and the class that receives the highest vote is assigned to the test



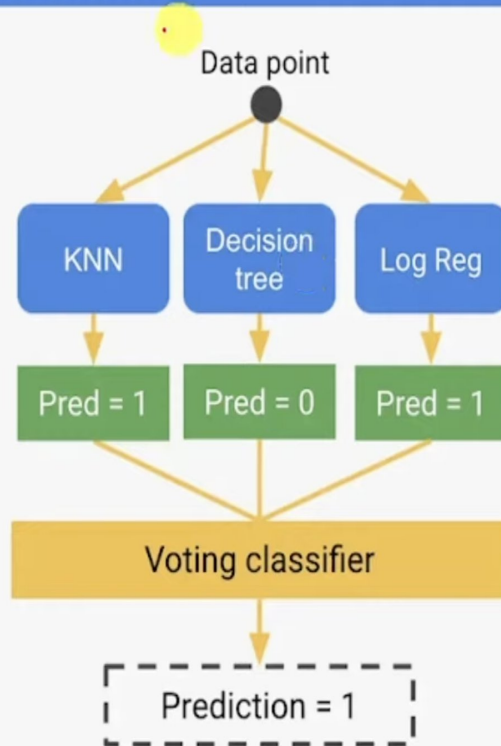
- By manipulating learning algorithm
 - Example: injecting randomness in the initial weights of ANN
- Many learning algorithms can be manipulated in such a way that applying the algorithm several times on the same training data will result in the construction of different classifiers.
- For example, an artificial neural network can change its network topology or the initial weights of the links between neurons.
- Similarly, an ensemble of decision trees can be constructed by injecting randomness into the tree-growing procedure. For example, instead of choosing the best splitting attribute at each node, we can randomly choose one of the top k attributes for splitting.
- The **first three approaches are generic methods** that are applicable to any classifier, whereas the fourth approach depends on the type of classifier used.
- The **base classifiers** for most of these approaches can be **generated sequentially** (one after another) or in **parallel** (all at once).



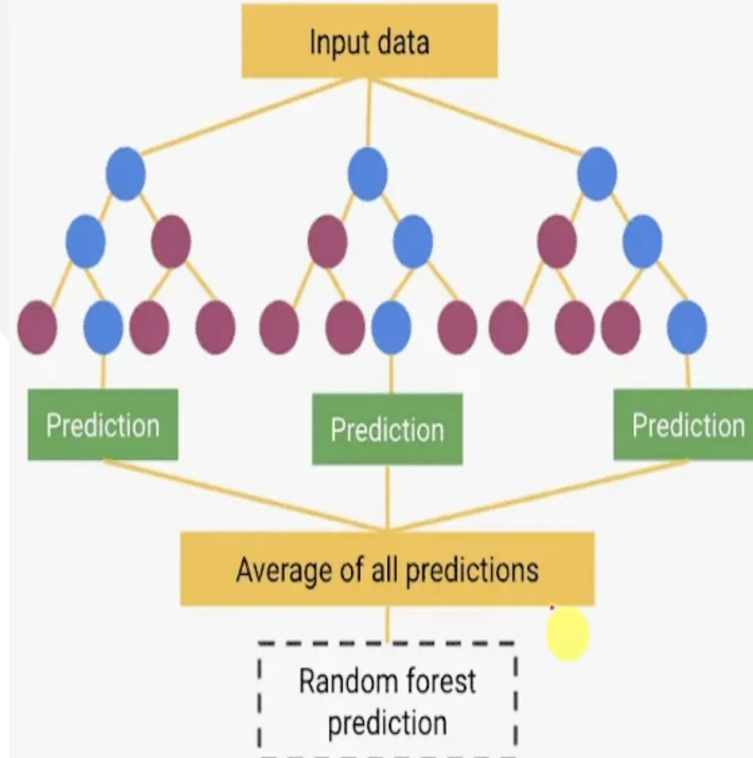
General Approach For Ensemble Learning

Go, change the world®

Hard voting



Random Forest



General Approach For Ensemble Learning

Go, change the world®



- Bootstrap sampling: sampling with replacement

Original Data	1	2	3	4	5	6	7	8	9	10
Bagging (Round 1)	7	8	10	8	2	5	10	10	5	9
Bagging (Round 2)	1	4	9	1	2	3	2	7	3	2
Bagging (Round 3)	1	8	5	10	5	5	9	6	3	7

- Build classifier on each bootstrap sample
- Probability of a training instance being selected in a bootstrap sample is:
 - ☐ $1 - (1 - 1/n)^n$ (n: number of training instances)
 - ☐ ~ 0.632 when n is large



• Bootstrap sampling: sampling with replacement

Original Data	1	2	3	4	5	6	7	8	9	10
Bagging (Round 1)	7	8	10	8	2	5	10	10	5	9
Bagging (Round 2)	1	4	9	1	2	3	2	7	3	2
Bagging (Round 3)	1	8	5	10	5	5	9	6	3	7

Round1:

Repeats: 8 (twice), 10 (three times), 5 (twice) •

Missing: 1, 3, 4, 6

Round2:

Repeats: 1 (twice), 2 (three times), 3 (twice) •

Missing: 5, 6, 8, 10

Round3:

Repeats: 5 (three times)

Missing: 2, 4

Each round trains a separate model on a **slightly different view** of the data. This introduces **diversity** among models, which is key to ensemble strength.

• **High variance models** (like decision trees) benefit most.

• When you **aggregate** predictions (e.g., majority vote), the

Ensemble becomes:

More **stable**

Less prone to **overfitting**

Better at **generalizing**



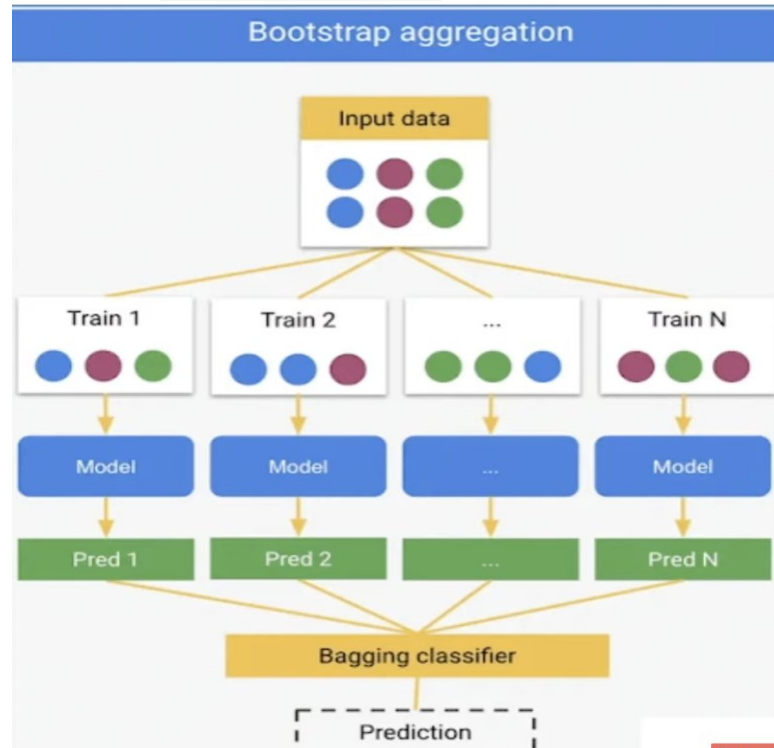
Algorithm 4.5 Bagging algorithm.

- 1: Let k be the number of bootstrap samples.
 - 2: **for** $i = 1$ to k **do**
 - 3: Create a bootstrap sample of size N , D_i .
 - 4: Train a base classifier C_i on the bootstrap sample D_i .
 - 5: **end for**
 - 6: $C^*(x) = \operatorname{argmax}_y \sum_i \delta(C_i(x) = y)$.
 $\{\delta(\cdot) = 1$ if its argument is true and 0 otherwise. $\}$
-



Bagging Algorithm

Go, change the world®





Create Multiple Bootstrap Samples

From the original training data **D** (size = n), we generate **B** bootstrap samples:

- Each sample is created by **sampling with replacement**
- Each sample has **n data points**
- Some samples contain repeated elements, and some original points are missing

Train a Base Model on Each Bootstrap Sample

A base learner (often a decision tree) is trained on each bootstrap dataset:

Model M_1 on Sample 1, Model M_2 on Sample 2, Model M_3 on Sample 3... up to M_B

Predict Using All Trained Models

For a new input **x**, each model makes its own prediction:

$$M_1(x), M_2(x), M_3(x), \dots, M_B(x)$$

So you get **B** predictions.

Aggregate the Predictions

For Regression:

Take the **average** of the predictions:

$$\hat{y} = \frac{1}{B} \sum_{i=1}^B M_i(x)$$

For Classification:

Use **majority voting**:

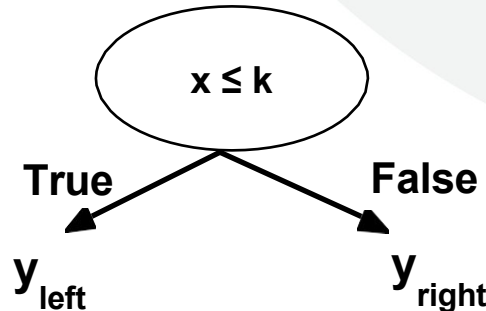
$\hat{y} = \text{most common label among } \{M_i(x)\}$

- Consider 1-dimensional data set:

Original Data:

x	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1
y	1	1	1	-1	-1	-1	-1	1	1	1

- Classifier is a decision stump (decision tree of size 1)
 - Decision rule: $x \leq k$ versus $x > k$
 - Split point k is chosen based on entropy





Bagging Example

Go, change the world®

Bagging Round 1:

x	0.1	0.2	0.2	0.3	0.4	0.4	0.5	0.6	0.9	0.9
y	1	1	1	1	-1	-1	-1	-1	1	1

$x \leq 0.35 \rightarrow y = 1$

$x > 0.35 \rightarrow y = -1$

Bagging Round 2:

x	0.1	0.2	0.3	0.4	0.5	0.5	0.9	1	1	1
y	1	1	1	-1	-1	-1	1	1	1	1

$x \leq 0.7 \rightarrow y = 1$

$x > 0.7 \rightarrow y = 1$

Bagging Round 3:

x	0.1	0.2	0.3	0.4	0.4	0.5	0.7	0.7	0.8	0.9
y	1	1	1	-1	-1	-1	-1	-1	1	1

$x \leq 0.35 \rightarrow y = 1$

$x > 0.35 \rightarrow y = -1$

Bagging Round 4:

x	0.1	0.1	0.2	0.4	0.4	0.5	0.5	0.7	0.8	0.9
y	1	1	1	-1	-1	-1	-1	-1	1	1

$x \leq 0.3 \rightarrow y = 1$

$x > 0.3 \rightarrow y = -1$

Bagging Round 5:

x	0.1	0.1	0.2	0.5	0.6	0.6	0.6	1	1	1
y	1	1	1	-1	-1	-1	-1	1	1	1

$x \leq 0.35 \rightarrow y = 1$

$x > 0.35 \rightarrow y = -1$



Bagging Example

Go, change the world®

Bagging Round 6:

x	0.2	0.4	0.5	0.6	0.7	0.7	0.7	0.8	0.9	1
y	1	-1	-1	-1	-1	-1	-1	1	1	1

$x \leq 0.75 \rightarrow y = -1$

$x > 0.75 \rightarrow y = 1$

Bagging Round 7:

x	0.1	0.4	0.4	0.6	0.7	0.8	0.9	0.9	0.9	1
y	1	-1	-1	-1	-1	1	1	1	1	1

$x \leq 0.75 \rightarrow y = -1$

$x > 0.75 \rightarrow y = 1$

Bagging Round 8:

x	0.1	0.2	0.5	0.5	0.5	0.7	0.7	0.8	0.9	1
y	1	1	-1	-1	-1	-1	-1	1	1	1

$x \leq 0.75 \rightarrow y = -1$

$x > 0.75 \rightarrow y = 1$

Bagging Round 9:

x	0.1	0.3	0.4	0.4	0.6	0.7	0.7	0.8	1	1
y	1	1	-1	-1	-1	-1	-1	1	1	1

$x \leq 0.75 \rightarrow y = -1$

$x > 0.75 \rightarrow y = 1$

Bagging Round 10:

x	0.1	0.1	0.1	0.1	0.3	0.3	0.8	0.8	0.9	0.9
y	1	1	1	1	1	1	1	1	1	1

$x \leq 0.05 \rightarrow y = 1$

$x > 0.05 \rightarrow y = 1$



- Summary of Trained Decision Stumps:

Round	Split Point	Left Class	Right Class
1	0.35	1	-1
2	0.7	1	1
3	0.35	1	-1
4	0.3	1	-1
5	0.35	1	-1
6	0.75	-1	1
7	0.75	-1	1
8	0.75	-1	1
9	0.75	-1	1
10	0.05	1	1

- Use majority vote (sign of sum of predictions) to determine class of ensemble classifier

Predicted Class	Round	x=0.1	x=0.2	x=0.3	x=0.4	x=0.5	x=0.6	x=0.7	x=0.8	x=0.9	x=1.0
	1	1	1	1	-1	-1	-1	-1	-1	-1	-1
	2	1	1	1	1	1	1	1	1	1	1
	3	1	1	1	-1	-1	-1	-1	-1	-1	-1
	4	1	1	1	-1	-1	-1	-1	-1	-1	-1
	5	1	1	1	-1	-1	-1	-1	-1	-1	-1
	6	-1	-1	-1	-1	-1	-1	-1	1	1	1
	7	-1	-1	-1	-1	-1	-1	-1	1	1	1
	8	-1	-1	-1	-1	-1	-1	-1	1	1	1
	9	-1	-1	-1	-1	-1	-1	-1	1	1	1
	10	1	1	1	1	1	1	1	1	1	1
	Sum	2	2	2	-6	-6	-6	-6	2	2	2
Sign	1	1	1	-1	-1	-1	-1	1	1	1	

- Bagging can also increase the complexity (representation capacity) of simple classifiers such as decision stumps



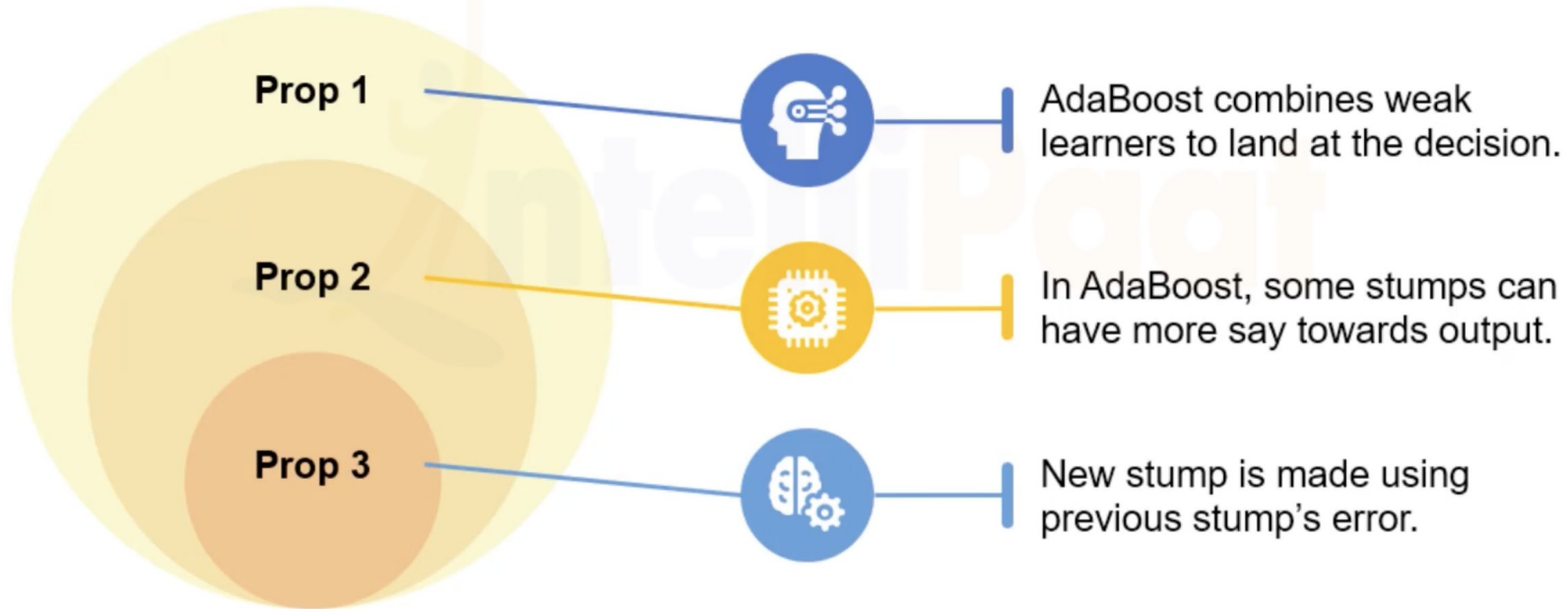
- An iterative procedure to adaptively change distribution of training data by focusing more on previously misclassified records
 - Initially, all N records are assigned equal weights (for being selected for training)
 - Unlike bagging, weights may change at the end of each boosting round



- Records that are wrongly classified will have their weights increased in the next round
- Records that are classified correctly will have their weights decreased in the next round

Original Data	1	2	3	4	5	6	7	8	9	10
Boosting (Round 1)	7	3	2	8	7	9	4	10	6	3
Boosting (Round 2)	5	4	9	4	2	5	1	7	4	2
Boosting (Round 3)	4	4	8	10	4	5	4	6	3	4

- Example 4 is hard to classify
- Its weight is increased, therefore it is more likely to be chosen again in subsequent rounds





Algorithm 4.6 AdaBoost algorithm.

- 1: $\mathbf{w} = \{w_j = 1/N \mid j = 1, 2, \dots, N\}$. {Initialize the weights for all N examples.}
 - 2: Let k be the number of boosting rounds.
 - 3: **for** $i = 1$ to k **do**
 - 4: Create training set D_i by sampling (with replacement) from D according to $\mathbf{w}$.
 - 5: Train a base classifier C_i on D_i .
 - 6: Apply C_i to all examples in the original training set, D .
 - 7: $\epsilon_i = \frac{1}{N} [\sum_j w_j \delta(C_i(x_j) \neq y_j)]$ {Calculate the weighted error.}
 - 8: **if** $\epsilon_i > 0.5$ **then**
 - 9: $\mathbf{w} = \{w_j = 1/N \mid j = 1, 2, \dots, N\}$. {Reset the weights for all N examples.}
 - 10: Go back to Step 4.
 - 11: **end if**
 - 12: $\alpha_i = \frac{1}{2} \ln \frac{1-\epsilon_i}{\epsilon_i}$.
 - 13: Update the weight of each example according to Equation 4.103.
 - 14: **end for**
 - 15: $C^*(\mathbf{x}) = \operatorname{argmax}_y \sum_{j=1}^T \alpha_j \delta(C_j(\mathbf{x}) = y)$.
-



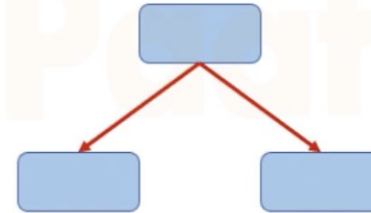
Adaboost Algorithm Example

Go, change the world®

Toy Data Sample

Chest Pain	Blocked Arteries	Patient Weight	Heart Disease	Sample Weights
Yes	Yes	205	Yes	1/8
No	Yes	180	Yes	1/8
Yes	No	210	Yes	1/8
Yes	Yes	167	Yes	1/8
No	Yes	156	No	1/8
No	Yes	125	No	1/8
Yes	No	168	No	1/8
Yes	Yes	172	No	1/8

Creation of the first stump is done by finding the variable **Chest Pain**, **Blocked Arteries**, or **Patient Weight** that does the best job of classifying the samples.



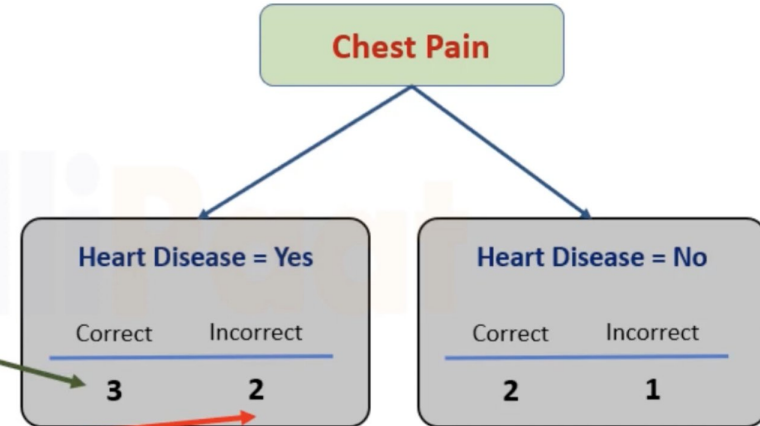


Adaboost Algorithm Example

Go, change the world®

Toy Data Sample

Chest Pain	Blocked Arteries	Patient Weight	Heart Disease	Sample Weights
Yes	Yes	205	Yes	1/8
No	Yes	180	Yes	1/8
Yes	No	210	Yes	1/8
Yes	Yes	167	Yes	1/8
No	Yes	156	No	1/8
No	Yes	125	No	1/8
Yes	No	168	No	1/8
Yes	Yes	172	No	1/8



We will begin by looking into how
feature chest pain helps in classification.



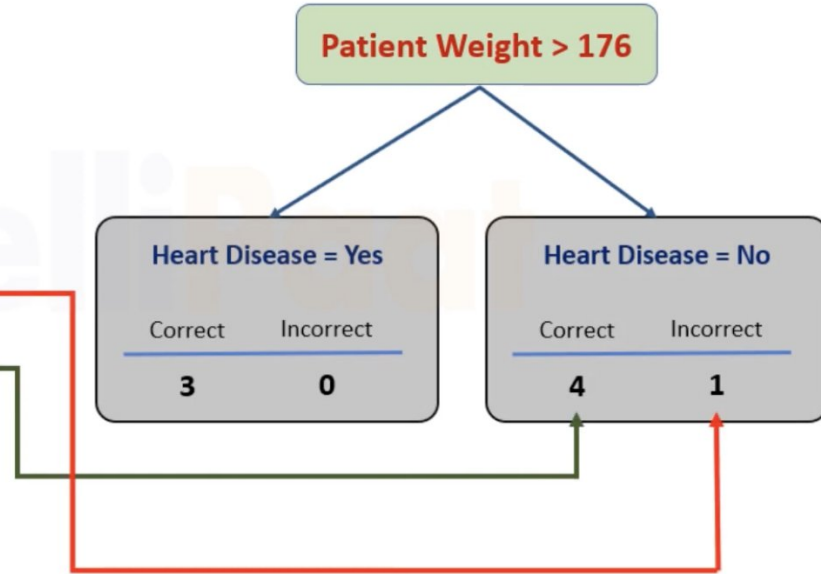
Adaboost Algorithm Example

Go, change the world®

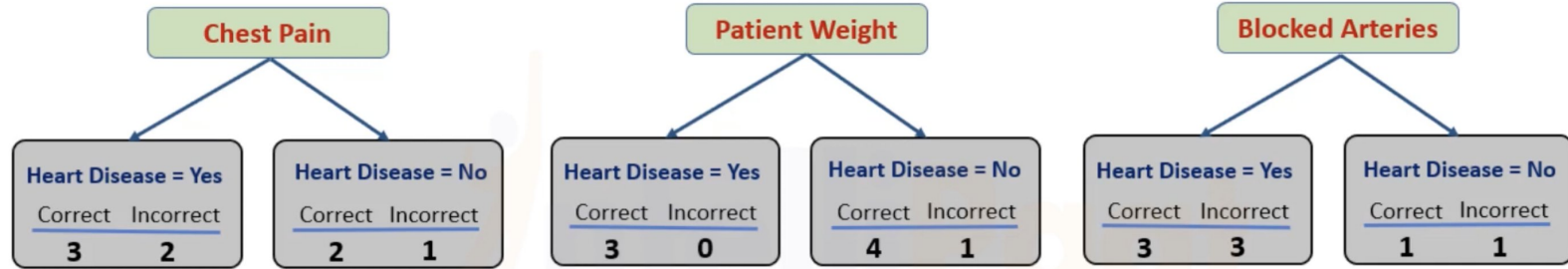
Toy Data Sample

Chest Pain	Blocked Arteries	Patient Weight	Heart Disease	Sample Weights
Yes	Yes	205	Yes	1/8
No	Yes	180	Yes	1/8
Yes	No	210	Yes	1/8
Yes	Yes	167	Yes	1/8
No	Yes	156	No	1/8
No	Yes	125	No	1/8
Yes	No	168	No	1/8
Yes	Yes	172	No	1/8

Creating Stump using feature **Patient Weight**.



After creating stumps, the next step is to calculate Gini Impurity value for each stump.



$$Gini (CP) = 1 - \left\{ \left(\frac{5}{8} \right)^2 + \left(\frac{3}{8} \right)^2 \right\} = 1 - \{0.39 + 0.14\} = 0.47$$

$$Gini (PW) = 1 - \left\{ \left(\frac{7}{8} \right)^2 + \left(\frac{1}{8} \right)^2 \right\} = 1 - \{0.76 + 0.01\} = 0.23$$

$$Gini (BA) = 1 - \left\{ \left(\frac{4}{8} \right)^2 + \left(\frac{4}{8} \right)^2 \right\} = 1 - \{0.25 + 0.25\} = 0.50$$



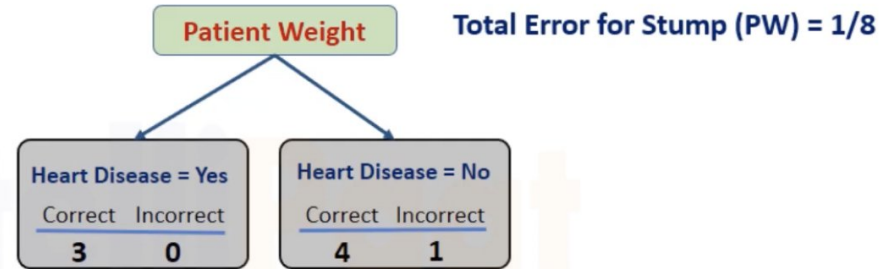
Adaboost Algorithm Example

Go, change the world®

Step – 3 : Calculate the **Amount of Say** selected stump has towards the output.

Toy Data Sample

Chest Pain	Blocked Arteries	Patient Weight	Heart Disease	Sample Weights
Yes	Yes	205	Yes	1/8
No	Yes	180	Yes	1/8
Yes	No	210	Yes	1/8
Yes	Yes	167	Yes	1/8
No	Yes	156	No	1/8
No	Yes	125	No	1/8
Yes	No	168	No	1/8
Yes	Yes	172	No	1/8



$$\text{Amount of Say} = \frac{1}{2} \log_e \left(\frac{1 - \text{Total Error}}{\text{Total Error}} \right)$$

Adaboost Algorithm Example

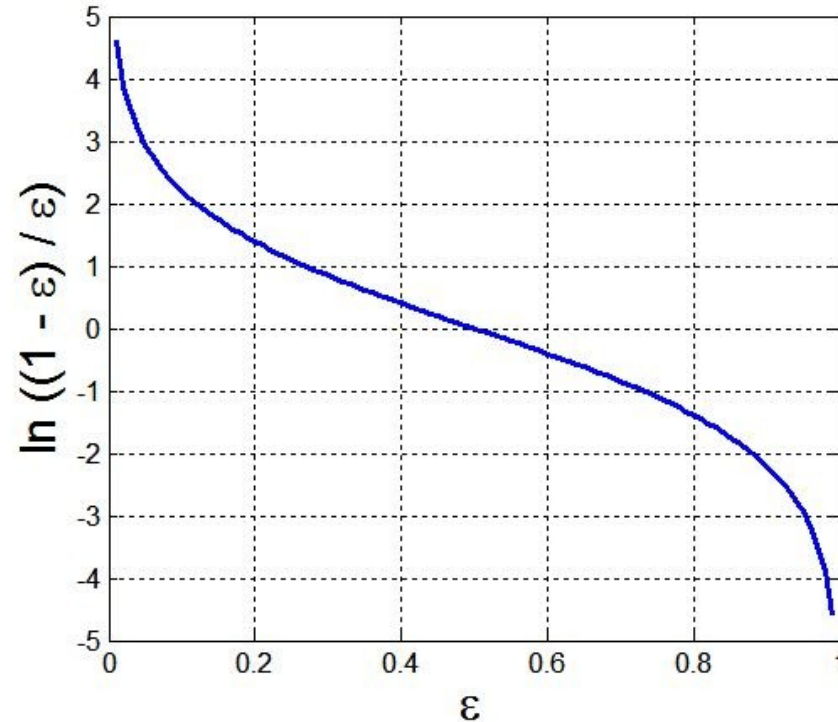
Go, change the world®

- Base classifiers: $C_1, C_2, \dots, C_T$
- Error rate of a base classifier:

$$\epsilon_i = \frac{1}{N} \sum_{j=1}^N w_j^{(i)} \delta(C_i(x_j) \neq y_j)$$

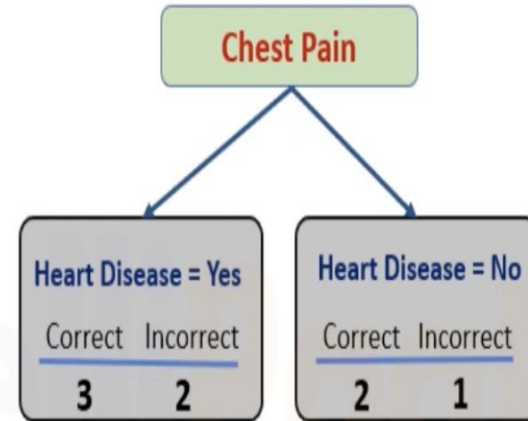
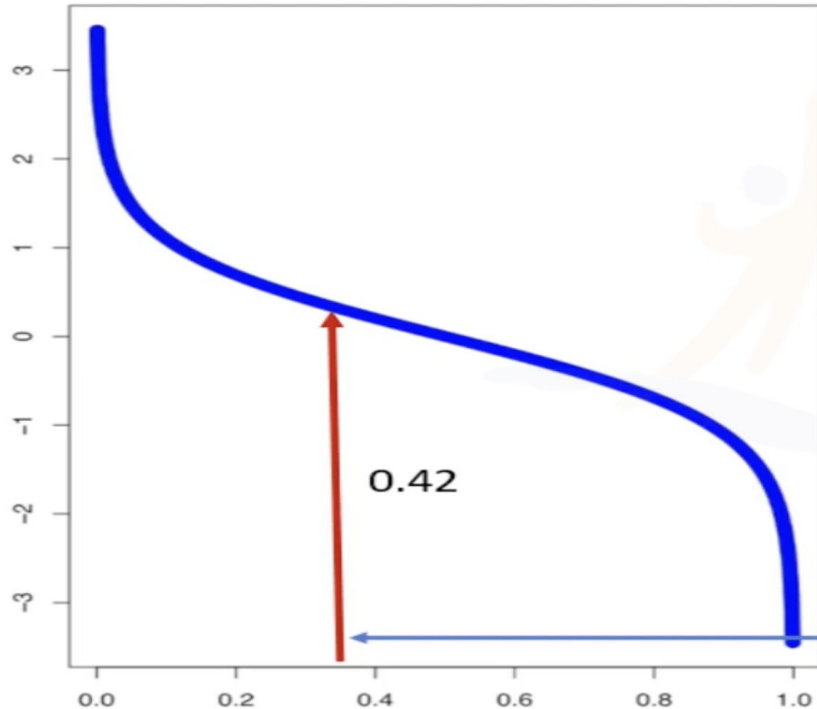
- Importance of a classifier:

$$\alpha_i = \frac{1}{2} \ln \left(\frac{1 - \epsilon_i}{\epsilon_i} \right)$$



Adaboost Algorithm Example

Go, change the world®



$$\text{Total Error (CP)} = 3 * 1/8 = 0.375$$

Adaboost Algorithm Example

Go, change the world®

Step – 4 : Create new stump by using error of previous stump.

Toy Data Sample

Chest Pain	Blocked Arteries	Patient Weight	Heart Disease	Sample Weights
Yes	Yes	205	Yes	1/8
No	Yes	180	Yes	1/8
Yes	No	210	Yes	1/8
Yes	Yes	167	Yes	1/8
No	Yes	156	No	1/8
No	Yes	125	No	1/8
Yes	No	168	No	1/8
Yes	Yes	172	No	1/8

New Sample Weight

$$= \text{Sample Weight}_{(Prev)} * e^{\text{Amount of Say}}$$

$$\text{New Weight} = 0.125 * e^{0.97} = 0.33$$

New Sample Weights for Rest of the Samples

$$= \text{Sample Weight}_{(Prev)} * e^{-\text{Amount of Say}}$$

$$\text{New Weight} = 0.125 * e^{-0.97} = 0.05$$

Adaboost Algorithm Example

Go, change the world®

Step – 4 : Create new stump by using error of previous stump.

Toy Data Sample

Chest Pain	Blocked Arteries	Patient Weight	Heart Disease	Updated Sample Weights
Yes	Yes	205	Yes	0.05
No	Yes	180	Yes	0.05
Yes	No	210	Yes	0.05
Yes	Yes	167	Yes	0.33
No	Yes	156	No	0.05
No	Yes	125	No	0.05
Yes	No	168	No	0.05
Yes	Yes	172	No	0.05

$\sum (Sample\ Weights)$ should always remain between 0 to 1. So we have to normalize this column.

Normalization Technique – Divide each sample weight by $\sum (Sample\ Weights)$

Adaboost Algorithm Example

Go, change the world®

Toy Data Sample

Chest Pain	Blocked Arteries	Patient Weight	Heart Disease	Normalized Sample Weights
Yes	Yes	205	Yes	0.07
No	Yes	180	Yes	0.07
Yes	No	210	Yes	0.07
Yes	Yes	167	Yes	0.49
No	Yes	156	No	0.07
No	Yes	125	No	0.07
Yes	No	168	No	0.07
Yes	Yes	172	No	0.07

Similarly we will keep on picking these samples. And we will create new bootstrapped data sample.

Chest Pain	Blocked Arteries	Patient Weight	Heart Disease
Yes	Yes	205	Yes
Yes	Yes	167	Yes
Yes	No	168	No
Yes	Yes	167	Yes
Yes	Yes	167	Yes
Yes	Yes	172	No
No	Yes	125	No
Yes	Yes	167	Yes

The sample with misclassification is repeated multiple times, because we want to emphasize on getting accurate result there.

Adaboost Algorithm Example

Go, change the world®

Step – 4 : Assign new sample weights, and create new stump for bootstrapped sample.

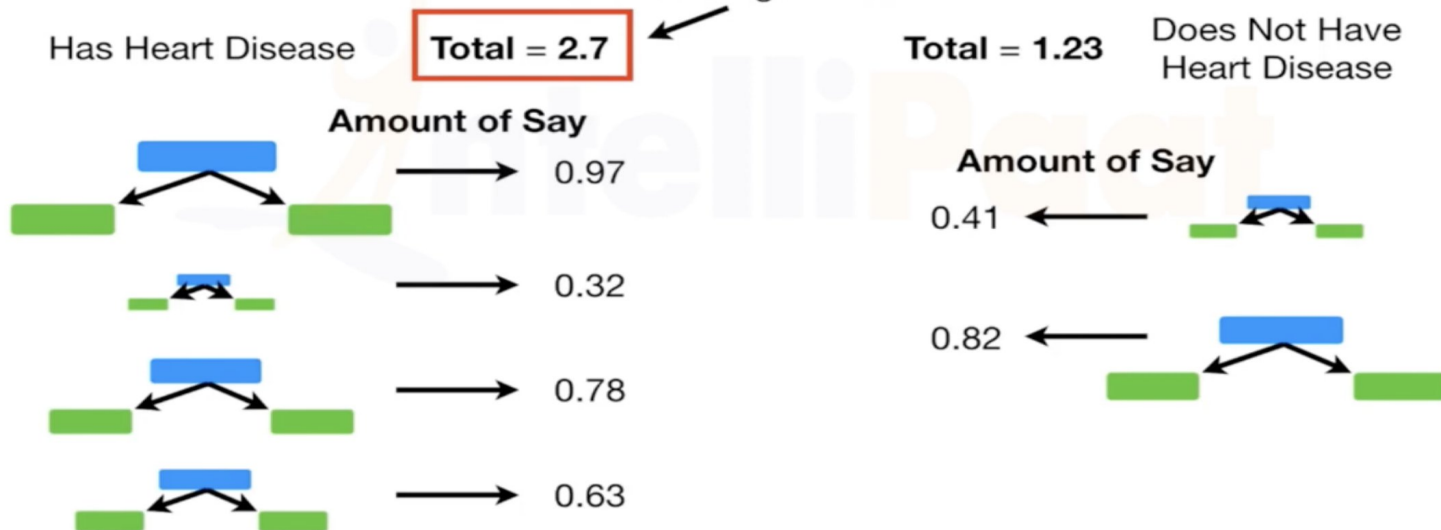
Bootstrapped Sample

Chest Pain	Blocked Arteries	Patient Weight	Heart Disease	Sample Weights
Yes	Yes	205	Yes	1/8
Yes	Yes	167	Yes	1/8
Yes	No	168	No	1/8
Yes	Yes	167	Yes	1/8
Yes	Yes	167	Yes	1/8
Yes	Yes	172	No	1/8
No	Yes	125	No	1/8
Yes	Yes	167	Yes	1/8

Since the misclassified samples are repeated multiple times, the new stump will focus more on getting these samples right. In other terms sample weight for misclassified sample is now – $4 * 1/8$.

Step – 5 : Combining stumps and making decision.

Ultimately, the patient is classified as **Has Heart Disease** because this is the larger sum.

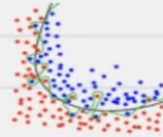


Advantages



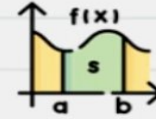
Good Accuracy

It focuses on misclassified instances, iteratively adjusting their weights.



Handles Complex Data

Can handle complex, non-linear data. Because it uses weak learners to learn the relations.



Feature Selection

The stump which has higher amount of say is selected.



Versatile & Easy

Can be combined with different weak classifiers such as SVM, DTs. And it is easy to implement.



Handles Imbalanced Data

By adjusting weights, minority classes can be given importance

Disadvantages

Go, change the world®

Disadvantages



Sensitive to Noisy Data

Assigns high weight to misclassified sample, so noisy data can have huge impact.



Computational Complexity

There is too much math behind algorithm. So cannot work with large data.



Lack of Transparency & Interpretability



Susceptibility to Biased Data

Possibility of a skewed model.



Parameter Sensitive

Tuning is a must.

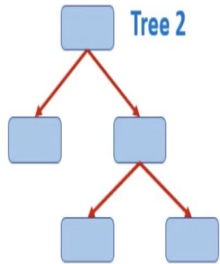
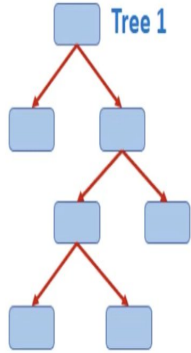
- Construct an ensemble of decision trees by manipulating training set as well as features
 - Use bootstrap sample to train every decision tree (similar to Bagging)
 - Use the following tree induction algorithm:
 - ◆ At every internal node of decision tree, randomly sample p attributes for selecting split criterion
 - ◆ Repeat this procedure until all leaves are pure (unpruned tree)

- Base classifiers are unpruned trees and hence are *unstable classifiers*
- Base classifiers are *decorrelated* (due to randomization in training set as well as features)
- Random forests reduce variance of unstable classifiers without negatively impacting the bias
- Selection of hyper-parameter p
 - Small value ensures lack of correlation
 - High value promotes strong base classifiers
 - Common default choices: $\sqrt{d}$, $\log_2(d + 1)$



Difference between AdaBoost and Random Forest

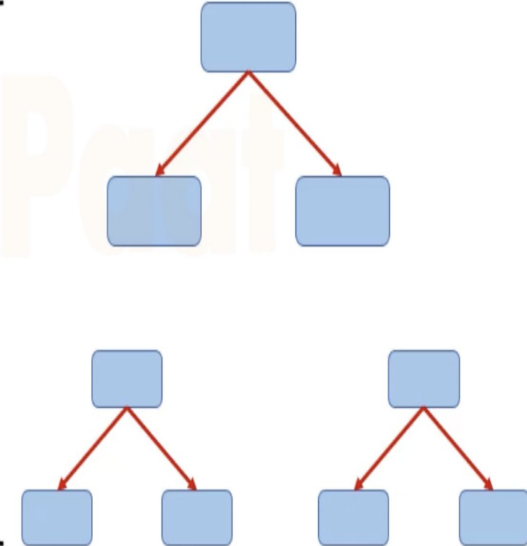
Go, change the world®



In case of **Random Forest**, different trees can have varying depth, there is **no predetermined maximum depth**.

In contrast to that, **AdaBoost** algorithm creates trees with **one parent node, and two leaves**. This structure is called as **Stump**.

AdaBoost is Forest of Stumps.





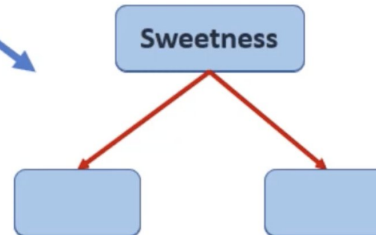
Difference between AdaBoost and Random Forest

Go, change the world®

Mango Type	Source	Sweetness
Alphonso	Maharashtra	4.5
Dasherri	Uttar Pradesh	3.8
Kesar	Maharashtra	3.7
Kesar	Maharashtra	4



In case of **Random Forest**, decision trees can use all the features of dataset to arrive at a decision.

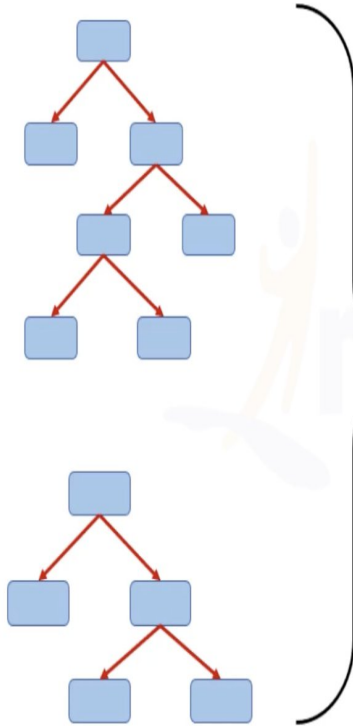


In case of **AdaBoost**, stumps can only use one feature, which makes them weak learners.



Difference between AdaBoost and Random Forest

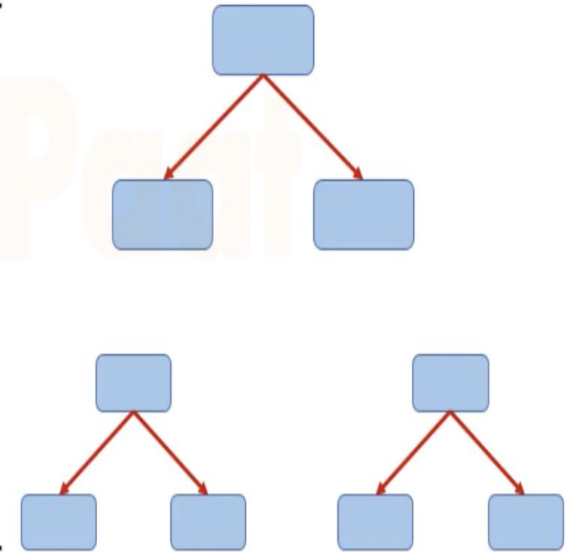
Go, change the world®



In case of **Random Forest**,
each tree has equal vote
towards final classification.

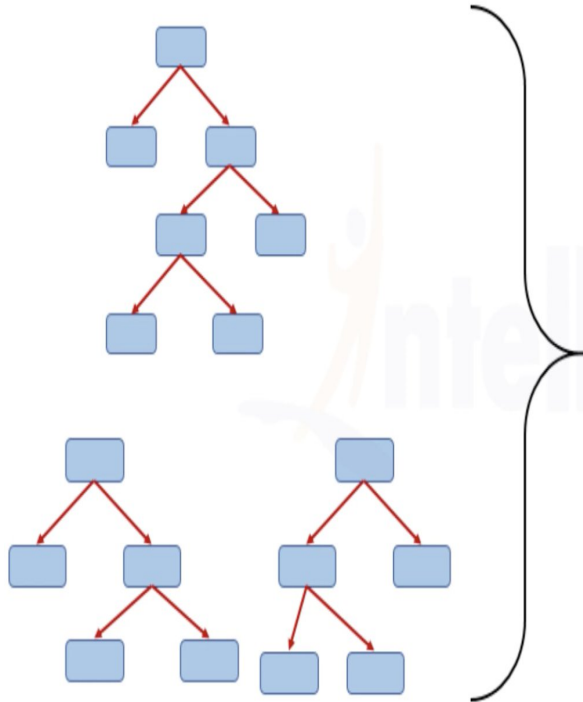
*Larger the stump,
more the amount
of say towards
final decision*

In case of **AdaBoost** algorithm **different
stumps get varied amount of say** towards
final classification.



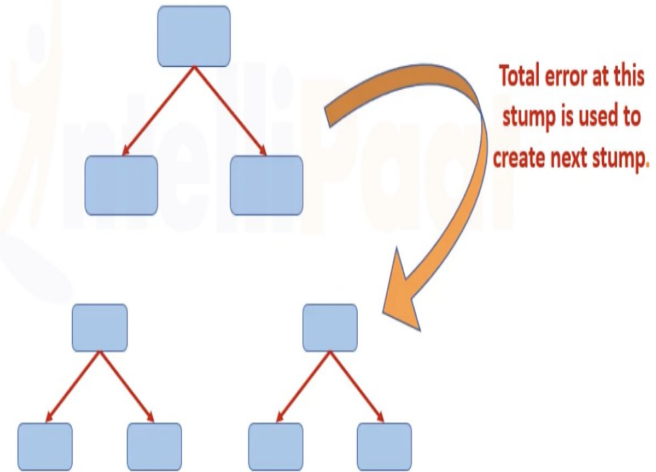
Difference between AdaBoost and Random Forest

Go, change the world®



In case of Random Forest,
each decision tree is made
independently of others.

In case of **AdaBoost** algorithm, the **order of stumps is quite important**. The error that the first stump makes influences how the second stump is made.



- Constructs a series of models
 - Models can be any predictive model that has a differentiable loss function
 - Commonly, trees are the chosen model
 - ◆ XGboost (extreme gradient boosting) is a popular package because of its impressive performance
- Boosting can be viewed as optimizing the loss function by iterative functional gradient descent.
- Implementations of various boosted algorithms are available in Python, R, Matlab, and more.